HTML to DITA-XML Conversion

Contents

Chapter 1. HTML to DITA-XML Conversion..... 6

Chapter 2. Getting Started..... 7

Purpose..... 7

Intended Audience..... 7

Introduction..... 8

Migration Prerequisites..... 9

Admonitions..... 10

Chapter 3. HTML Typical Package Structure..... 11

HTML Typical Folder Structure..... 11

HTML Typical Page Structure..... 12

HTML Body Content..... 13

Chapter 4. HTML Index..... 14

Scenario 1 — Index file title becomes root Ditamap title..... 15

Scenario 2 — ZIP folder name becomes root Ditamap title..... 15

Chapter 5. HTML Metadata..... 17

Scenario 1 — HTML metadata mapped to <prolog>..... 18

Chapter 6. Headings..... 20

Scenario 1 — Headings applied correctly..... 21

Scenario 2 — Heading level hierarchy..... 22

Scenario 3 — No <h1> present..... 23

Scenario 4 — Consecutive headings with no body content..... 23

Chapter 7. Paragraphs..... 25

Scenario 1 — Paragraph in a concept topic..... 26

Scenario 2 — Paragraph with mixed inline character styles..... 26

Chapter 8. Inline Formatting..... 28

Scenario 1 — Bold used for general emphasis..... 29

Scenario 2 — Bold used for a UI element inside a procedure..... 29

Scenario 3 — Code element used for inline code....30
Scenario 4 — Code element used for a file path....31
Scenario 5 — Bold used for Menu navigation sequence....31
Chapter 9. Lists....33
Ordered Lists (Numbered)....33
Scenario 1 — Consecutive Numbered list as a Procedure....34
Scenario 2 — Consecutive Numbered list as a Workflow....35
Scenario 7 — Two-Level Nested Numbered List (sub steps) in Procedure....36
Scenario 8 — Two-Level Nested Numbered List in Workflow....37
Scenario 9 — Three-level deep nested ordered list in Workflow....38
Scenario 10 — Three-level deep nested Numbered list in Procedure (multi-level sub steps)....39
Scenario 11 — Three-level deep nested Numbered list in Procedure (Sub-steps and Options)....40
Scenario 12 — Admonition Between Numbered List in a Procedure....42
Scenario 13 — Admonition Between Numbered List in a Workflow....42
Scenario 14 — Paragraph Between Numbered List in a Procedure....43
Scenario 15 — Paragraph Between Numbered List in a Workflow....44
Scenario 16 — Block Image Between Numbered List in a Procedure....45
Scenario 17 — Block Image Between Numbered List in a Workflow....46
Scenario 18 — Table Between Numbered List in a Workflow....47
Scenario 19 — Table Between Numbered List in a Procedure....49
Scenario 20 — Bullet List between Numbered List in a Procedure....50
Scenario 21 — Nested Bullet List Between Numbered List in a Procedure....51
Unordered Lists....54
Scenario 1 — Consecutive bullet items....55
Scenario 2 — Two-level nested bullet list....56
Scenario 3 — Three-level deep nested bullet list....57
Scenario 4 — Block image placed between bullet list items....58
Scenario 5 — Paragraph Between a Bullet list item....59

Scenario 6 — Table embedded inside a list item..... 60
Scenario 7 — Admonition between Bullet List..... 61
Term List (Definition List)..... 62
Scenario 1 — Native <dl> element..... 64
Scenario 2 — list with bold term and separator pattern..... 65
Chapter 10. Images and Graphics..... 67
Scenario 1 — Image Size is Smaller than 50 Pixels..... 69
Scenario 2 — Image Size is Larger than 50 Pixels..... 69
Scenario 3 — Image Size is Larger than 100 Pixels..... 70
Scenario 4 — Inline image within correct wrap mode..... 71
Scenario 5 — Image with caption..... 71
Scenario 6 — Multiple images placed side by side..... 72
Scenario 7 — Image inside a numbered list item..... 73
Scenario 8 — Image inside a bulleted list item..... 74
Scenario 9 — Image Linked via External URL Rather than a Local Package Reference..... 75
Scenario 10 — Image Placed Inside a Positioned (Text Box Equivalent)..... 75
Scenario 11 — Image with Floating Label Elements Layered on Top..... 76
Scenario 12 — Image with an Alt Text..... 77
Chapter 11. Tables..... 78
Scenario 1 — Standard multi-column table..... 79
Scenario 2 — Table with a <caption> element..... 80
Scenario 3 — Table with no header row..... 82
Scenario 4 — Table with merged cells..... 83
Scenario 5 — Nested table (table inside a table cell)..... 84
Scenario 6 — List inside a table cell..... 86
Scenario 7 — Image inside a table cell..... 86
Scenario 8 — Table spanning multiple pages..... 87
Scenario 9 — Multiple consecutive tables with no paragraph between them..... 89
Scenario 10 — 2-column table in an API reference section..... 91

Chapter 12. Admonitions....92

Scenario 1 — Image placed inside an admonition.... 94

Scenario 2 — Admonition inside a table cell....95

Scenario 3 — Bold-label admonition pattern....95

Chapter 13. Cross-References and Hyperlinks.... 97

Scenario 1 — Internal link.... 98

Scenario 2 — External URLs.... 99

Scenario 3 — Hyperlink with non-descriptive display text....99

Chapter 14. Math and Equations.... 101

Scenario 1 — Block equation.... 102

Scenario 2 — Inline equation....103

Scenario 3— Equation in a table cell.... 104

Scenario 4 — Equation in a list item....105

Scenario 5 — Equation in a procedure step....106

Chapter 15. Additional Reference/Appendix.... 108

Topic Type Classification....108

Concept Topic.... 109

Task Topic.... 111

Reference Topic.... 113

Troubleshooting Topic....116

Generic Topic.... 119

Index....a

Chapter 1. HTML to DITA-XML Conversion

Best Practices Guide

Best Practices Guide

Preparing HTML Files for Accurate DITA-XML Conversion

Chapter 2. Getting Started

This section describes the purpose and audience of this guide as well as the prerequisites for HTML conversion to DITA-XML. It covers the mandatory and optional HTML constructs, document preparation requirements, and the scenarios that affect conversion readiness. The converter and AI Agent use these structures to detect document hierarchy, classify topic types, infer metadata, and generate accurate DITA maps and topic files.

Purpose

This guide provides mandatory best practices for anyone who creates or prepares HTML source content

This guide provides mandatory best practices for anyone who creates or prepares HTML source content for conversion to DITA-XML using the metR DocMigration HTML to DITA converter. The converter is a modular microservice with an embedded AI Agent that reasons and enriches DITA elements in document structure before transforming them into standards-compliant DITA-XML.

Following these guidelines ensures that:

- The converter correctly identifies topic boundaries, heading hierarchies, and content types.
- The AI Conversion Agent classifies topics accurately as concept, task, reference, or troubleshooting.
- The generated DITA-XML output is clean and complete, requiring minimal post-conversion cleanup.
- Metadata and Ditamap hierarchies are detected reliably.

Intended Audience

This guide is intended for the following audiences:

This guide is intended for the following audiences:

Table 1. Intended audience and guide applicability

Audience	How this guide applies
Technical writers and document authors	Primary authoring before submitting HTML files for conversion.
DITA architects and consultants	Reference when reviewing or auditing source documents before migration projects.

Note:
You must have basic working knowledge of HTML heading levels, links, tables, lists, images, and metadata. This guide assumes that you can inspect HTML in a browser and, when needed, in source view.

Introduction

The metR HTML to DITA converter does not rely on visual formatting alone. It reads

The metR HTML to DITA converter does not rely on visual formatting alone. It reads structural information from HTML elements and semantic markup. When those elements are applied correctly, the converter and AI Agent can:

- Generate a root Ditamap and child Ditamaps from the index file, ZIP folder name, or document title.
- Detect topic boundaries from heading levels and establish the Ditamap hierarchy.
- Classify topic types: concept (descriptive), task (procedural), reference (tabular or lookup), or troubleshooting.
- Convert paragraphs, lists, tables, images, links, inline formatting, and code blocks to valid DITA elements.
- Infer metadata — short descriptions, keywords, index terms, author, and revision history — from `<head>` meta tags and structured body elements.
- Flag issues such as missing short descriptions, invalid links, duplicate IDs, orphan topics, and improper nesting for human review.

The converter reads the following HTML constructs to accomplish this. Each is covered in a dedicated section of this guide:

Table 2. Mandatory HTML constructs for conversion

#	Mandatory HTML Construct	Role in Conversion
1	Index file title (`index.html` or `index.htm`)	Becomes the root Ditamap `<title>` element. If no index file exists, the ZIP folder name is used instead.
2	Heading 1	Drives the Ditamap hierarchy and primary topic boundaries.
3	Heading 2 – Heading N	Each heading generates a separate child Ditamap and a corresponding DITA topic, nested according to heading level.
4	Paragraphs	Maps to `<p>` elements inside topic body elements.

Table 2. Mandatory HTML constructs for conversion (continued)

#	Mandatory HTML Construct	Role in Conversion
5	Tables	Converts to `<simpletable>` or to a CALS `<table>` + `<tgroup>`
6	Images and Graphics	Converts to DITA `<image>` elements with optional `<alt>` text. Block images are wrapped in `<fig>`.
7	Ordered lists	Maps to DITA `<ol><li>` or to `<steps><step><cmd>` structures.
8	Unordered lists	Maps to DITA `<ul><li>` elements or `<prereq>` in task topics.

Table 3. Optional HTML constructs for conversion

#	Optional HTML Construct	Role in Conversion
9	Index file hierarchy	Used to build the root Ditamap navigation hierarchy; not converted as a regular DITA topic.
10	Metadata	Maps to Ditamap metadata and topic `<prolog>` elements including `<author>`, `<critdates>`, and `<keywords>`.
11	Character formatting (bold, italic, monospace)	Maps to bold `<b>`, italic `<i>`, or inline code `<codeph>` elements.
12	Admonitions (note, warning, tip, important, caution)	Maps to the DITA `<note>` element with the appropriate type attribute.
13	Cross - References and Hyperlinks	Maps to DITA cross-reference `<xref>` element with appropriate scope and format attributes.
14	Math and Equations	Maps to `<equation-block>` for block equations or inline `<math>` for inline equations, with MathML content preserved.

Migration Prerequisites

Before submitting a document to the metR HTML to DITA converter, verify the following prerequisites:

Before submitting a document to the metR HTML to DITA converter, verify the following prerequisites:

1. The HTML files are complete and available in the expected folder structure.
2. The root folder contains the main index, if available, along with the HTML files.
3. Each HTML file has a clear heading hierarchy.
4. Heading levels are not used only for visual styling.

5. Images are available in the package and referenced correctly.
6. Internal and external links are valid.
7. Tables use real HTML table markup and are not simulated with spaces, tabs, or images.
8. Code samples use `<code>` or `<pre>`where possible.

> **Tip:**
> Before conversion, review the HTML in a browser and in source view. Confirm that the visible content and the HTML structure match.

Admonitions

This guide uses four types of admonitions to call out information that needs extra attention.

This guide uses four types of admonitions to call out information that needs extra attention. Each one signals a different level of importance, as explained below.

Table 4. Admonition label types and uses

Label	Used for
NOTE	Supplementary information, clarifications, or good-to-know context that doesn't block the reader from proceeding
WARNING	Critical information the reader must understand to avoid damage, data loss, safety risk, or severe consequences
TIP	Optional best-practice advice or a shortcut that improves the outcome but isn't required
IMPOR-TANT	Critical information the reader must understand to get an accurate conversion — not option-al, but not a risk of damage or data loss

Chapter 3. HTML Typical Package Structure

This section describes the typical structure of an HTML package submitted for conversion to DITA-XML, including folder organization, file naming, and the placement of media assets. It covers the standard HTML page structure the converter expects, focusing on how both `<head>` metadata and `<body>` content is read and mapped. The converter uses these structures to identify the entry point, build the root and child Ditamap hierarchy, determine topic boundaries from heading levels, and generate accurate DITA maps and topic files.

HTML Typical Folder Structure

A typical HTML package contains the following:

A typical HTML package contains the following:

```
HTML Package (ZIP)
├── index.html          → creates the root Ditamap hierarchy (if present)
├── topic1.html         → creates a child Ditamap and DITA topic
├-- topic2.html         → creates a child Ditamap and DITA topic
├── media/
|   ├── image1.png
|   └── image2.svg
├── css/
|   └── styles.css      → ignored
└── scripts/
└── script.js           → ignored
```

The converter reads the HTML constructs given in the following bullet list:

- **Index file (index.html , optional).** Creates the root Ditamap hierarchy and serves as the entry point for the converted documentation.
- **HTML files (.html or .htm).** The converter creates a child Ditamap and a primary DITA topic from each file.
- **Images and media.** The converter maps referenced assets to DITA `<image>` elements.

The converter reads the package structure to create the root Ditamap folder hierarchy, identifies the main entry point, preserves relative paths for images and media, and generates child Ditamaps based on individual HTML files. Submit HTML content as a ZIP file.



Tip:
Organize HTML files in a logical folder structure before conversion. The converter preserves the folder hierarchy in the DITA output.

HTML Typical Page Structure

Each HTML file follows standard HTML5 document structure:

Each HTML file follows standard HTML5 document structure:

```
<html>
  <head>
    <title>Page Title</title>
    <meta name="author" content="Documentation Team">
    <meta name="created" content="2026-01-15">
  </head>
  <body>
    [Content appears here]
    <footer>
      [Footer content]
    </footer>
  </body>
</html>
```

The converter focuses on two key areas:

1. `<head>` **metadata**. The converter maps this content to DITA `<prolog>` elements such as author, created date, and revised date
2. `<body>` **content.** The converter maps this content to DITA topic body elements, including paragraphs, lists, tables, and images.



Warning:
The converter ignores footer content by default, as well as CSS, JavaScript, navigation menus, cookie banners, advertisements, and hidden elements. If your HTML file contains important content in the footer, move it to the `<body>` section before conversion.

HTML Body Content

The `<body>` element contains semantic content that maps to DITA topics and topic body elements. The converter processes elements in document order and uses the heading hierarchy to determine topic boundaries and nesting levels.

The converter extracts the HTML package, analyzes metadata and content structure, applies heading-based hierarchy rules, and generates the corresponding Ditamaps and topics.



Warning:
If the HTML uses visual formatting instead of semantic structure, the converter may append content to the wrong topic, create an incorrect hierarchy, or generate DITA that requires manual cleanup.

Chapter 4. HTML Index

This section covers the Table of Contents and Root Index — the navigation structure that maps the publication hierarchy and serves as the converter's primary entry point for building the root DITA map — and two scenarios: a root index file present in the package, and a missing root index file. The converter reads the root index to construct the DITA map hierarchy and topic references; when no root index is present, it falls back to the package folder name or filename as the root map title.

A table of contents or root index is the navigation structure of a publication. It declares the document's complete hierarchy in one place — the titles, sequence, and parent-child relationships of all sections, chapters, and topics — and provides the entry point from which readers and systems navigate the entire deliverable.

For the publishing system, the table of contents or root index is the structural anchor of the entire output — every generated navigation artifact, including the PDF bookmark panel, the HTML5 navigation tree, the WebHelp TOC panel, and the DITA map hierarchy, derives from this structure.



Note:
The converter reads it to build the DITA map hierarchy and then discards it — the publishing transform regenerates the navigation structure at build time from the assembled DITA map, ensuring it always reflects the actual content rather than a manually maintained list that can fall out of sync.

In a well-structured HTML package, the table of contents is provided by the navigation structure in the index.html or index.htm file. Each navigation link references an HTML document within the publication, and the order and nesting of these links establish the hierarchy of the generated documentation.

The converter reads the navigation structure in the index file to generate the root Ditamap hierarchy. The table of contents itself is not converted into a standalone DITA topic. Instead, each navigation link becomes a corresponding `<mapref>` or `<topicref>` in the generated root Ditamap, preserving the organization of the original HTML publication.



Important:
Ensure that the table of contents accurately reflects the intended publication hierarchy and that every navigation link references a valid HTML file. Missing, incorrect, or incomplete navigation links result in an incomplete Ditamap hierarchy and may exclude topics from the generated publication.

Scenario 1 — Index file title becomes root Ditamap title

Consider an HTML package where the ZIP folder is named Printer_Docs. The index file contains

Consider an HTML package where the ZIP folder is named Printer_Docs. The index file contains a clear title, Printer User Guide, inside the `<head>` tag.

Table 5. Title tag becomes ditamap title

HTML Input	DITA Output

ZIP folder name: `Printer_Docs`
Index file: `index.html`
Index file title: `<head><title>Printer User Guide</title></head>`
HTML files: `installation.html`, `maintenance.html`
	`<map>`
	`  <title>Printer User Guide</title>`
	`  <mapref href="installation.ditamap"/>`
	`  <mapref href="maintenance.ditamap"/>`
	`</map>`

Result: When an index file contains a valid HTML `<title>` element, the converter uses that title (Printer User Guide) as the root Ditamap title. The ZIP folder name is used only when a suitable title is not present in the HTML package.

Scenario 2 — ZIP folder name becomes root Ditamap title

Consider an HTML package without an index file, and the ZIP folder has a generic

Consider an HTML package without an index file, and the ZIP folder has a generic or unclear name.

Table 6. ZIP folder name becomes title

HTML Input	DITA Output

ZIP folder name: `html_output`
Index file: not present
HTML files: `installation.html`, `maintenance.html`
	`<?xml version="1.0" encoding="UTF-8"?>`
	`<!DOCTYPE map PUBLIC "-//OASIS//DTD Ditamap//EN" "map.dtd">`
	`<map>`
	`  <title>HTML_output</title>`
	`  <mapref href="installation_0.ditamap"/>`
	`  <mapref href="maintenance.ditamap"/>`
	`</map>`

Result: When the HTML package contains no index file, the converter uses the ZIP folder name as the root Ditamap title. The HTML files in the package are converted into corresponding Ditamaps and topics and are referenced from the root map.

Chapter 5. HTML Metadata

This section covers meta tags and structured metadata elements in HTML — the metadata fields that identify, attribute, and classify a document independently of its body content — and one scenario: `<head>` meta tags populated in the source file and mapped during conversion. The converter reads each recognized meta tag and maps it to the corresponding DITA `<prolog>` element, making the document's authorship, dates, and classification data available to downstream publishing and search tools.

Metadata is structured information about a document, separate from its body content. It records attributes such as authorship, creation and revision dates, and classification tags. This information is what allows downstream publishing systems, content management platforms, and search indexes to reliably identify, track, and retrieve the document.

In HTML documents, metadata lives in two places: the `<head>` section as `<meta>` tags, and occasionally in the document `<body>` as CMS-generated structured elements such as Confluence's `<div class="page-metadata">` block. Both carry the same categories of information but in different forms that the converter must recognize and extract from each location.

The converter and AI Agent map this metadata to the DITA `<prolog>` element in the generated topic, making it available to downstream publishing tools, content management systems, and search indexes.

Table 7. HTML elements mapped to DITA

HTML Element / Meta Tag	DITA Output
`<meta name="author">` or `<span class="author">`	`<prolog><author>`
`<meta name="created">` or text parsed from Created by...	`<prolog><critdates><created>`
`<meta name="last-modified">` or text parsed from last modified on...	`<prolog><critdates><revised>`
`<meta name="keywords">`	`<metadata><keywords>`
`<meta name="description">`	`<shortdesc>` or `<metadata><othermeta name="description">`
`<meta name="company">`	`<othermeta name="company">`
`<meta name="category">`	`<othermeta name="category">`
`<title>` or `<h1>`	`<title>` or `<mainbooktitle>`

Note:
Metadata not found in the HTML source, or in an unrecognized format, is not included in the DITA output. Add it manually after conversion or ensure the source uses a supported format before submitting.

Scenario 1 — HTML metadata mapped to <prolog>

Scenario 1 — HTML metadata mapped to <prolog> Consider an HTML document where the author

Consider an HTML document where the author has provided metadata using standard `<meta>` tags inside the `<head>` section, covering author, creation and revision dates, keywords, and a short description.

Table 8. Metadata mapped to topic prolog

HTML Input	DITA Output
<head> <title>Configuring Network Settings</title> <meta name="author" content="XYZ"/> <meta name="created" content="2025-03-14"/> <meta name="last-modified" content="2025-11-02"/> <meta name="keywords" content="network, configuration, setup"/> <meta name="description" content="Steps to configure the device network settings."/> <meta name="company" content="XYZ Corp"/> </head>	<concept id="configuring_network_settings"> <title>Configuring Network Settings</title> <prolog> <author>XYZ</author> <critdates> <created date="2025-03-14"/> <revised modified="2025-11-02"/> </critdates> <metadata> <keywords> <keyword>network</keyword> <keyword>configuration</keyword> <keyword>setup</keyword> </keywords> <othermeta name="company" content="XYZ Corp"/> </metadata> </prolog> <shortdesc>Steps to configure the device network settings.</shortdesc> <conbody>

Table 8. Metadata mapped to topic prolog (continued)

HTML Input	DITA Output
	... </conbody> </concept>

Result: Each recognized `<meta>` tag is mapped to its corresponding DITA `<prolog>` element — **author** maps to `<author>`, **created** and **last-modified** map to `<critdates><created>` and `<critdates><revised>`, and **keywords** maps to a list of `<keyword>` elements inside `<metadata>`. The **description** meta tag maps to `<shortdesc>`, and **company** maps to `<othermeta name="company">` since it has no dedicated DITA prolog element. The `<title>` element supplies the **topic title**.

Chapter 6. Headings

This section covers the Heading style in HTML — its role as the primary structural mechanism that organizes a document's content into a named, hierarchical sequence of sections — and four scenarios: headings applied in correct sequential order, a heading level skipped in the hierarchy, a top-level heading absent from the document, and consecutive headings with no body content between them. The converter reads each heading level to construct the DITA map hierarchy, generating sub-maps, topic files, and topic type classifications from the heading sequence and the content that follows each heading.

A **Heading**style marks the beginning of a named section of content and assigns it a rank in the document hierarchy. It is the primary structural mechanism of any technical document — headings divide continuous content into discrete, named units, establish the parent-child relationships between those units, and create the navigational backbone that readers and publishing systems depend on.

The heading hierarchy is expressed through levels: a top-level heading (Heading 1) names a major section; each lower-level heading (Heading 2 through Heading N) names a subsection within its parent. This ranked sequence carries the same structural meaning regardless of the format it is expressed. The level communicates the section's position in the hierarchy; the text communicates its subject.



Note:
Only a semantically marked heading — applied through the document's native heading mechanism — creates a genuine section boundary that a converter, search engine, or publishing system can detect and act on.

In well-formed HTML, headings are applied sequentially without skipping levels. An `<h1>` is followed by one or more `<h2>` elements, each `<h2>` may contain one or more `<h3>` elements, and so on.

The converter reads the heading elements within the `<body>` of each HTML file to determine how many DITA topic files to generate, how to title and nest them in the Ditamap, and what topic type each one receives. It then analyzes the content that follows each heading, such as paragraphs, lists, and tables, to classify the resulting topic as Concept, Task, Reference, Troubleshooting, or Generic.

The following table shows how each HTML heading element maps to a DITA construct:

Table 9. Heading elements mapped to DITA

HTML Element	Maps To	Generates
`<h1>` with content	Parent DITAMAP	The `<Heading 1>` text becomes both the Ditamap title and the primary topic title. Generates a .ditamap file and a corresponding .dita topic file.
`<h1>` (no content)	Navigation container	A `<topichead>` element in the parent map. No DITA topic file is generated.
`<h2>`	Child DITAMAP	A child .ditamap file and a corresponding .dita topic file, referenced from the parent map.
`<h3>`	Grandchild DITAMAP	A nested .ditamap file and a corresponding .dita topic file, referenced from the child map.
`<h4>`–`<h6>`	Lower-level DITAMAPs	Additional nested .ditamap and .dita topic files at deeper nesting levels.

Warning:
Never use `<strong>` or `<b>` elements to simulate headings. Bold text styled to look like a heading is not recognized as a structural marker. The content that follows it is added to the preceding topic. Use heading elements to define topic boundaries.

The following scenarios demonstrate how different HTML heading structures are interpreted during HTML-to-DITA conversion and how they affect the generated Ditamap and topic hierarchy:

Scenario 1 — Headings applied correctly

Consider an HTML document with the following heading structure: <heading 1> "Printer Installation" containing <heading

Consider an HTML document with the following heading structure: `<heading 1>` "Printer Installation" containing `<heading 2>` "Printer Prerequisites" (concept) and `<heading 2>` "Install the Printer Driver" (task).

Table 10. Headings applied correctly

HTML Input	DITA Output
`<h1 id="printer-installation">Printer Installation</h1>`	`<map id="printer_installation">`

Table 10. Headings applied correctly (continued)

HTML Input	DITA Output

`<h2 id="printer-prerequisites">Printer Prerequisites</h2>`
`<h2 id="install-printer-driver">Install the Printer Driver</h2>`
	`<title>Printer Installation</title>`
	`<topicref href="printer_prerequisites.dita"/>`
	`<topicref href="install_printer_driver.dita"/>`
	`</map>`

Result: The `<heading1>` section (**Installation**) creates a dedicated DITAMAP. Each `<heading 2>` section becomes a separate child dita topic referenced through a `<topicref>` element. The converter classifies topic types according to the content pattern following each heading:

- **Prerequisites**
- **Install on Linux**

This structure preserves both the document hierarchy and the semantic meaning of the content.

Scenario 2 — Heading level hierarchy

Consider an HTML document where an <h1> element groups related <h2> sections but contains no

Consider an HTML document where an `<h1>` element groups related `<h2>` sections but contains no direct body content of its own. All paragraphs, tables, and images appear under `<h2>` or deeper heading levels.

Table 11. Heading level hierarchy

HTML Input	DITA Output

`<h1 id="installation">Installation</h1>`
`<h2 id="prerequisites">Prerequisites</h2>`
`<h3 id="system-requirements">System Requirements</h3>`
`<h2 id="installing-the-server">Installing the Server</h2>`
	`<map id="m001_installation">`
	`<title>Installation</title>`
	`<topichead navtitle="Installation">`
	`<mapref href="t001_prerequisites.ditamap"/>`
	`<mapref href="t002_system_requirements.ditamap"/>`
	`<mapref href="t003_installing_the_server.ditamap"/>`
	`</topichead>`
	`</map>`

Result: No .dita topic file is created for **Heading 1: Installation** because it has no body content. The converter maps it to a `<topichead>` element and nests the related **Heading 2** and **Heading 3** topics under it.

Scenario 3 — No <h1> present

Scenario 3 — No <h1> present Consider an HTML document where the author starts the

Consider an HTML document where the author starts the content with `<h2>` and never applies `<h1>` anywhere in the document. Similarly, in another case, the author may skip both `<h1>` and `<h2>` and start directly with `<h3>`.

Table 12. No h1 present

HTML Input	DITA Output

`<h2 id="installation">Installation</h2>`
`<h3 id="install-on-linux">Install on Linux</h3>`
`<h2 id="configuration">Configuration</h2>`
`<h2 id="troubleshooting">Troubleshooting</h2>`
`(No <h1> present)`
	`<map id="root_map">`
	`<title>Printer User Guide</title>`
	`<mapref href="m001_installation.ditamap" format="ditamap"/>`
	`<mapref href="m002_install_on_linux.ditamap" format="ditamap"/>`
	`<mapref href="m003_configuration.ditamap" format="ditamap"/>`
	`<mapref href="m004_troubleshooting.ditamap" format="ditamap"/>`
	`</map>`

Result: The converter promotes the highest heading level present to the `<h1>` role. Each promoted heading generates a .ditamap and a corresponding .dita topic file. The root Ditamap title comes from the index file title or ZIP folder name, not the HTML content.



Note:
The converter promotes the highest heading level present when no `<h1>` is found, ensuring content is still converted. For best results, always begin each HTML file with a single `<h1>` to make the hierarchy explicit.

Scenario 4 — Consecutive headings with no body content

Consider an HTML document where multiple consecutive headings at any level have no body content

Consider an HTML document where multiple consecutive headings at any level have no body content between them.

Table 13. Consecutive headings, no body content

HTML Input	DITA Output

`<h1 id="installation">Installation</h1>`
`<h2 id="prerequisites">Prerequisites</h2>`
`<h3 id="system-requirements">System Requirements</h3>`
`<p>The system requires 4 GB RAM.</p>`
`(No body content under <h1> or <h2>)`
	`<map id="m001_installation">`
	`  <title>Installation</title>`
	`  <topichead navtitle="Installation">`
	`  <topichead navtitle="Prerequisites">`
	`  <mapref href="t001_system_requirements.ditamap"/>`
	`  </topichead>`
	`  </topichead>`
	`</map>`

Result: Each heading with no body content directly beneath it is converted to a `<topichead>` navigation container rather than a .dita topic file. The first heading that has body content beneath it — `<h3>` (System Requirements) — generates a .ditamap and a corresponding .dita topic file, and is referenced in the parent map using a `<mapref>` element.



Note:
Add at least a short introductory paragraph directly beneath any heading that should become a real .dita topic rather than a navigation-only `<topichead>`. Without body content, the converter has no material from which to generate a topic file.

Chapter 7. Paragraphs

Abstract: This section covers the Paragraph style in HTML — its role as the primary

Abstract: This section covers the Paragraph style in HTML — its role as the primary carrier of explanatory, introductory, and transitional prose in any technical document — and four scenarios: a paragraph in a concept topic, a paragraph containing mixed inline character styles, a paragraph placed before the first heading, and a code paragraph. The converter maps each paragraph to a `<p>` element inside the classified topic body, extracts the `<shortdesc>` based on the paragraph context, and maps recognized inline character styles to their corresponding DITA inline elements.

Paragraphsare the fundamental unit of prose communication in any technical publication. They carry three types of content as described below:

- Explanatory content that defines concepts and terms.
- Introductory content that sets the context for the structured elements that follow such as list, workflows, tables, procedures, and images.
- Transitional content that connects sections and chapters through the document's logic.

This prose content is essential not only for readability but for building complete technical context, search discoverability and structural completeness.



Tip:
A topic consisting only of a list or a table, with no surrounding prose, is a valid but an incomplete piece of technical communication. It is recommended to add the context to such structured elements.

In well-structured HTML, introductory paragraphs establish the context for a topic before lists, tables, or images present supporting information. Inline elements such as `<strong>`, `<em>`, `<code>`, and other recognized HTML tags provide semantic emphasis within the surrounding paragraph without changing its structural role.

The converter maps each HTML `<p>` element to a DITA `<p>` element within the generated topic body, whether `<conbody>`, `<taskbody>`, `<refbody>`, `<troublebody>`, or `<body>`. The first introductory paragraph beneath a heading is also used to populate the generated `<shortdesc>`; element where applicable. Recognized inline HTML elements are converted to their equivalent DITA inline elements while preserving their semantic meaning. Empty paragraphs used only for visual spacing are ignored during conversion.

Important:
Paragraphs should always appear within a heading-delimited section. A `<p>` element placed before the first heading is treated as pre-heading content and is processed according to a separate conversion rule. Use paragraphs to present meaningful content rather than visual spacing, and rely on recognized inline markup instead of manual formatting to convey emphasis.

The following scenarios demonstrate how different paragraph structures are interpreted during HTML-to-DITA conversion:

Scenario 1 — Paragraph in a concept topic

Consider an HTML section titled "Product Parts Location" followed by two <p> elements describing the

Consider an HTML section titled "Product Parts Location" followed by two `<p>` elements describing the product.

Table 14. Paragraph in a concept topic

HTML Input	DITA Output

`<h2>Product parts location</h2>`
`<p>Use this section to identify the parts on the printer.</p>`
`<p>The printer has three main areas: front, inside, and back.</p>`
	`<concept id="product_parts_location">`
	`  <title>Product Parts Location</title>`
	`  <conbody>`
	`  <p>Use this section to identify the parts on the printer.</p>`
	`  <p>The printer has three main areas: front, inside, and back.</p>`
	`  </conbody>`
	`</concept>`

Result: Both `<p>` elements are mapped to `<p>` elements inside `<conbody>` in document order. The first `<p>` element is also extracted as `<shortdesc>`.

Scenario 2 — Paragraph with mixed inline character styles

Consider an HTML `<p>` element that contains a mix of bold, italic, and monospace formatting applied to individual words or phrases within the same paragraph. The converter maps each recognized inline HTML element to the corresponding DITA inline element within the same `<p>` element.

Table 15. Paragraph with mixed inline character styles

HTML Input	DITA Output
`<p>Press <strong>OK</strong> to save, or run <code>setup.sh</code> using <em>administrator</em> privileges.</p>`	`<p>Press <b>OK</b> to save, or run <codeph>setup.sh</codeph> using <i>administrator</i> privileges.</p>`

Result: The converter preserves all inline formatting within a single `<p>` element. Each recognized inline HTML element is mapped to its DITA equivalent:

- `<strong>` → `<b>`,
- `<em>` → `<i>`,
- `<code>` → `<codeph>`.

For a full list of inline element mappings, see the Inline Formatting *(on page 28)* section.

Chapter 8. Inline Formatting

This section covers Inline Formatting in HTML — the inline formatting attributes that assign semantic meaning to individual words or phrases within a paragraph — and five scenarios: bold for general emphasis, bold for a UI element inside a procedure, monospace for inline code, monospace for a file path, and bold for a menu navigation sequence. The converter reads each named character style or font family to determine the appropriate DITA inline element; where named styles are absent, content pattern recognition refines the mapping.

Inline Formattingare attributes applied to individual words or phrases within a paragraph. For example,

- A menu path or button label is bold to identify UI screen text as a named interface element.
- A file path or command is monospace to identify it as executable or navigable content.

Marking these distinctions visually — bold, italic, or monospace — communicates meaning to readers. Translation memory tools, conditional processing engines, content reuse frameworks, and accessibility processors all require typed markup to treat each kind of text correctly in publishing systems.

The HTML-to-DITA converter evaluates each recognized HTML inline element within its surrounding paragraph and maps it to the appropriate DITA inline element. General emphasis is preserved using `<b>` or `<i>`, while recognized patterns within `<code>` or `<span>` elements are converted to more specific DITA elements such as `<codeph>`, `<filepath>`, `<varname>`, `<cmdname>`, or `<menucascade>`. This semantic mapping preserves the meaning of inline content while keeping the surrounding paragraph intact.

Inline formatting in HTML is applied to individual words or phrases within a `<p>` element using inline elements such as `<strong>`, `<em>`, `<code>`, and `<span>`.

The following table summarizes how recognized HTML inline elements are mapped during HTML-to-DITA conversion:

Table 16. Inline elements mapped to DITA

HTML Inline Element	DITA Inline Element
`<strong>` (general emphasis)	`<b>`
`<strong>` (UI label in procedure)	`<uicontrol>`
`<em>`	`<i>`
`<code>` (inline code term)	`<codeph>`
`<code>` (file path pattern)	`<filepath>`

Table 16. Inline elements mapped to DITA (continued)

HTML Inline Element	DITA Inline Element
`<code>` (variable name pattern)	`<varname>`
`<code>` (command name pattern)	`<cmdname>`
`<span>` with menu navigation pattern (`>`separator)	`<menucascade><uicontrol>`...

Important:
Use semantic inline elements to identify emphasis, user interface controls, commands, code, file paths, variables, and menu navigation. Avoid using inline formatting solely for visual presentation, as unrecognized or non-semantic markup may be converted to plain text, reducing the semantic value of the generated DITA output.

Scenario 1 — Bold used for general emphasis

Consider an HTML paragraph that reads: "The Server Management module is used to manage various

Consider an HTML paragraph that reads: "The Server Management module is used to manage various virtual servers." The term "Server Management" is wrapped in a `<strong>` element for general emphasis.

Table 17. Bold used for general emphasis

HTML Input	DITA Output

`<p>`
`  The <strong>Server Management</strong> module is used to`
` manage various virtual servers.`
`</p>`
	`<p>`
	`    The <b>Server Management</b> module is used to`
	` manage various virtual servers.`
	`</p>`

Result: The converter maps `<strong>` used for general emphasis to `<b>` within the surrounding `<p>` element.

Scenario 2 — Bold used for a UI element inside a procedure

Consider a procedural step that reads: "Click the Save button." The word "Save" is wrapped

Consider a procedural step that reads: "Click the **Save** button." The word "Save" is wrapped in `<strong>` because it represents a UI button label.

Table 18. Strong used for UI element

HTML Input	DITA Output

`<ol>`
`  <li>`
`  Click the <strong>Save</strong> button.`
`  </li>`
`</ol>`
	`<taskbody>`
	`  <steps>`
	`  <step>`
	`  <cmd>`
	`  Click the <uicontrol>Save</uicontrol> button.`
	`  </cmd>`
	`  </step>`
	`  </steps>`
	`</taskbody>`

Result: The converter recognizes that the `<strong>` element represents a UI label within a procedural step and maps it to `<uicontrol>` instead of `<b>`. A `<strong>` element inside an `<ol><li>` within a task topic is treated as a UI label and mapped to `<uicontrol>`. The same `<strong>` element inside a descriptive `<p>` in a concept topic would be mapped to `<b>`.

Scenario 3 — Code element used for inline code

Consider an HTML paragraph that contains a shell script name wrapped in a `<code>` element.

Table 19. Code element used for inline code

HTML Input	DITA Output

`<ol>`
`  <li>`
`  Run <code>deploy.sh</code> using elevated`
` privileges.`
`  </li>`
`</ol>`
	`<taskbody>`
	`  <steps>`
	`  <step>`
	`  <cmd>`
	`  Run <codeph>deploy.sh</codeph> using elevated`
	` privileges.`
	`  </cmd>`
	`  </step>`

Table 19. Code element used for inline code (continued)

HTML Input	DITA Output
	</steps> </taskbody>

Result: The `<code>` element representing an inline code term or script name is mapped to `<codeph>`.

Scenario 4 — Code element used for a file path

Consider an HTML paragraph that contains a file system path wrapped in a `<code>` element.

Table 20. Code element used for a file path

HTML Input	DITA Output
 The application configuration file is located at <code>C:\Program Files\PrinterApp\config\settings.xml</code>. 	<conbody> The application configuration file is located at <filepath>C:\Program Files\PrinterApp\config\settings.xml</filepath>. </conbody>

Result: The converter detects that the `<code>` element contains a file system path pattern and maps it to `<filepath>` instead of `<codeph>`.

Note:
A `<code>` element whose content matches a file-system path pattern, such as directory separators (\ or /), drive letters, directory names, or file names, is mapped to `<filepath>`. All other `<code>` content is mapped to `<codeph>` by default.

Scenario 5 — Bold used for Menu navigation sequence

Consider an HTML paragraph or procedural step that contains a menu navigation path such as

Consider an HTML paragraph or procedural step that contains a menu navigation path such as "File → Save As", where each menu label is wrapped in a `<strong>` element or the entire path is wrapped in a `<span>` element with a `>` separator between menu labels.

Table 21. Bold used for Menu navigation sequence

HTML Input	DITA Output

`<ol>`
`  <li>`
`  Go to`
`  <span class="menucascade">`
`  <strong>File</strong> > <strong>Save As</strong>`
`  </span>.`
`  </li>`
`</ol>`
	`<taskbody>`
	`  <steps>`
	`  <step>`
	`  <cmd>`
	`  Go to`
	`  <menucascade>`
	`  <uicontrol>File</uicontrol>`
	`  <uicontrol>Save As</uicontrol>`
	`  </menucascade>.`
	`  </cmd>`
	`  </step>`
	`  </steps>`
	`</taskbody>`

Result: The converter detects the menu navigation pattern — multiple `<strong>` elements separated by a `>` character within a `<span>` — and maps the sequence to a `<menucascade>` structure containing nested `<uicontrol>` elements.

Chapter 9. Lists

This section covers Lists in HTML — the primary structure for presenting discrete, parallel items in a defined or co-equal sequence — and introduces two subsections: Ordered Lists and Unordered Lists. The converter reads the list type, nesting level, and topic context to determine whether to map each list to `<ol>`, `<ul>`, `<steps>`, `<substeps>`, `<choices>`, or `<dl>` in the DITA output.

Listscommunicate sequences, collections, and term-definition pairs that prose paragraphs cannot organize with equal clarity. They are the primary structure for presenting discrete, parallel items — steps in a procedure, items in a requirements checklist, and options in a configuration section.

A list communicates to the reader that its items share a structural relationship: either sequence matters and the order is meaningful, or the items are co-equal and interchangeable in position.

The converter evaluates the HTML list type together with its surrounding context — topic classification, position relative to other content, and nesting — to generate the appropriate DITA list structure. Ordered lists, unordered lists, and term lists are each governed by their own mapping rules, described in the sections that follow.

> **Important:**
> Always use native HTML list elements (ol and ul) to represent structured information. Do not simulate lists using manually typed numbers, bullet characters, hyphens, or spacing within paragraphs, as these patterns are treated as plain text during conversion.

Ordered Lists (Numbered)

This section covers Ordered Lists in HTML — items presented in a defined sequence where position is significant — and twenty-one scenarios grouped across six categories: consecutive and discontinuous numbered lists as procedures and workflows, single-item numbered lists, two-level and three-level nested numbered lists, and interrupted numbered lists where an admonition, paragraph, image, table, or bullet list appears between items. The converter maps each ordered list to `<ol><li>` in concept and reference topics, or to `<steps><step><cmd>` in task topics, based on the heading classification and content context.

An **ordered list** presents items in a defined sequence where the position of each item is significant. The ordered lists are universally used for step-by-step procedures, installation instructions, and ranked items.

For the reader, a numbered list communicates that sequence is contractual — item 2 depends on item 1 being completed, and performing the items out of order produces an incorrect or unsafe result. It is also

a navigational reference that lets a reader say "I am on step 4" and return to the correct position after an interruption.

The list level is controlled by the Increase and Decrease Indent buttons, determines nesting depth: level 1 is the top level, level 2 is the first nested level, level 3 is the second, and so on. Sometimes they are also defined by their tab indentation. The list level assigned by list controls is what drives the nesting structure in the DITA output.

In well-structured HTML, ordered lists are created using native `<ol>` and `<li>` elements. Nested ordered lists represent subordinate sequences, while introductory paragraphs, images, tables, and notes provide supporting information without changing the sequential relationship between list items. A new `<ol>` element beginning after an existing list — rather than continuing it — signals a restarted or independent sequence rather than a continuation of the prior list.

The converter evaluates both the HTML list structure and the classified DITA topic type when processing an ordered list. Within Concept, Reference, and Generic topics, an HTML `<ol>` is converted to a DITA `<ol>`. Within a Task topic, the same `<ol>` is converted into `<steps>`, with each `<li>` mapped to a `<step>` containing a `<cmd>`. The surrounding context determines how supporting content associated with each list item is represented in the generated DITA structure.

The following scenarios demonstrate how different HTML ordered list patterns are interpreted during HTML-to-DITA conversion:

Scenario 1 — Consecutive Numbered list as a Procedure

Consider an <h2> called "Install on Linux", the body contains a numbered list which indicates

Consider an `<h2>` called "Install on Linux", the body contains a numbered list which indicates step-by-step procedure.

Table 22. Numbered list as a procedure

HTML Input	DITA Output

`<h2 id="install-on-linux">Install on Linux</h2>`
`<ol>`
`  <li>Download the installer.</li>`
`  <li>Run the setup wizard.</li>`
`  <li>Restart the service.</li>`
`</ol>`
	`<task id="install_on_linux">`
	`  <title>Install on Linux</title>`
	`  <taskbody>`
	`  <steps>`
	`  <step>`
	`  <cmd>Download the installer.</cmd>`

Table 22. Numbered list as a procedure (continued)

HTML Input	DITA Output
	</step> <step> <cmd>Run the setup wizard.</cmd> </step> <step> <cmd>Restart the service.</cmd> </step> </steps> </taskbody> </task>

Result: The converter classifies "Install on Linux" as a task topic because the heading contains an action verb, and an `<ol>` list is the primary body content. Each `<li>` element is mapped to a `<step><cmd>` element inside `<steps>`.

Scenario 2 — Consecutive Numbered list as a Workflow

Consider an <h2> heading "Deployment Environment Lifecycle" classified as a concept topic.

Consider an `<h2>` heading "Deployment Environment Lifecycle" classified as a concept topic. The body contains an introductory `<p>` element followed by an `<ol>` list of workflow stages.

Table 23. Numbered list as a Workflow

HTML Input	DITA Output
<h2 id="deployment-lifecycle"> Deployment Environment Lifecycle </h2> The following is the deployment environment lifecycle. - Development Environment - Staging Environment	<concept id="deployment_environment_lifecycle"> <title>Deployment Environment Lifecycle</title> <conbody> The following is the deployment environment lifecycle. - Development Environment - Staging Environment

Table 23. Numbered list as a Workflow (continued)

HTML Input	DITA Output
- UAT Environment - Production Environment 	- UAT Environment - Production Environment </conbody> </concept>

Result: The converter writes an `<ol>` inside `<conbody>` with each `<li>` preserved, because the heading and introductory paragraph signal descriptive concept content rather than a procedure.

Scenario 7 — Two-Level Nested Numbered List (sub steps) in Procedure

Consider a numbered list with a second tier of numbered sub-items beneath selected parent items

Consider a numbered list with a second tier of numbered sub-items beneath selected parent items as part of the instructions or procedure.

Table 24. Nested sub-steps in a procedure

HTML Input	DITA Output
<h2 id="configure-server">Configure the Server</h2> Configure the server. - Open the configuration file. - Set the port value. - Restart the service. 	<task id="configure_server"> <title>Configure the Server</title> <taskbody> <steps> <step> <cmd>Configure the server.</cmd> <substeps> <substep> <cmd>Open the configuration file.</cmd> </substep> <substep> <cmd>Set the port value.</cmd> </substep> </substeps> </step> <step>

Table 24. Nested sub-steps in a procedure (continued)

HTML Input	DITA Output
	<cmd>Restart the service.</cmd> </step> </steps> </taskbody> </task>

Result: Two level numbered lists are preserved in `<taskbody>` as a `<substeps>` block inside `<steps>`.

Scenario 8 — Two-Level Nested Numbered List in Workflow

Consider a numbered list with a second tier of numbered sub-items beneath selected parent items

Consider a numbered list with a second tier of numbered sub-items beneath selected parent items as part of the workflow.

Table 25. Nested numbered list in workflow

HTML Input	DITA Output
<h2 id="document-approval-workflow">D ocument Approval Workflow</h2> Documents progress through several stages before they are approved for publication. The following workflow illustrates the overall approval lifecycle. Authoring Draft creation	<concept id="document_approval_workflow"> <title>Document Approval Workflow</title> <conbody> Documents progress through several stages before they are approved for publication. The following workflow illustrates the overall approval lifecycle. Authoring - Draft creation - Initial self-review Review

Table 25. Nested numbered list in workflow (continued)

HTML Input	DITA Output
- Initial self-review Review - Technical review - Editorial review 	- Technical review - Editorial review </conbody> </concept>

Result: Two level numbered lists are preserved in `<conbody>` as a second `<ol>` block nested inside the parent `<ol>` block.

Scenario 9 — Three-level deep nested ordered list in Workflow

Consider a concept topic where an <ol> list has three levels of nesting representing phases,

Consider a concept topic where an `<ol>` list has three levels of nesting representing phases, sub-phases, and sub-items.

Table 26. Three level nested ordered list in workflow

HTML Input	DITA Output
<h2 id="release-workflow">Release Workflow</h2> Planning Requirement review - Product input - Documentation input	<concept id="release_workflow"> <title>Release Workflow</title> <conbody> Planning Requirement review

Table 26. Three level nested ordered list in workflow (continued)

HTML Input	DITA Output
             Execution Build validation - QA approval - Release approval 	- Product input - Documentation input Execution Build validation - QA approval - Release approval </conbody> </concept>

Result: The converter maps all three numbering levels as nested ordered lists — Level 0 becomes the parent `<ol>`, Level 1 becomes a nested `<ol>`, and Level 2 becomes a further nested `<ol>` inside the Level 1 `<li>`.

Scenario 10 — Three-level deep nested Numbered list in Procedure (multi-level sub steps)

Consider a task topic where the first two levels represent procedural actions, and the third

Consider a task topic where the first two levels represent procedural actions, and the third level represents additional ordered supporting details for a substep.

Table 27. Three level nested sub-steps in procedure

HTML Input	DITA Output
<h2 id="configure-the-server">Configure the Server</h2> Update the configuration file. Open the configuration file. - Locate the server section. - Review the existing values. - Restart the server. 	<task id="configure_the_server"> <title>Configure the Server</title> <taskbody> <steps> <step> <cmd>Update the configuration file.</cmd> <substeps> <substep> <cmd>Open the configuration file.</cmd> <info> - Locate the server section. - Review the existing values. </info> </substep> </substeps> </step> <step> <cmd>Restart the server.</cmd> </step> </steps> </taskbody> </task>

Result: The converter maps Level 0 to `<steps><step><cmd>`. Level 1 maps to `<substeps><substep><cmd>` and Level 2 maps to `<info><ol><li>`.

Scenario 11 — Three-level deep nested Numbered list in Procedure (Sub-steps and Options)

Consider a procedure where the first two levels represent procedural actions, and the third level

Consider a procedure where the first two levels represent procedural actions, and the third level presents mutually exclusive options the user must choose from, rather than supporting reference details. In HTML, this third level is authored as a `<ul>` nested inside the `<ol>`.

Table 28. Three-level nested sub-steps and options

HTML Input	DITA Output

`<h2 id="configure-the-server">Configure the Server</h2>`
`<ol>`
`  <li>`
`  Configure the server.`
`  <ol>`
`  <li>`
`  Select the authentication mode.`
`  <ul>`
`  <li>Local Authentication</li>`
`  <li>LDAP Authentication</li>`
`  <li>SAML Authentication</li>`
`  </ul>`
`  </li>`
`  </ol>`
`  </li>`
`  <li>Restart the service.</li>`
`</ol>`
	`<task id="configure_the_server">`
	`  <title>Configure the Server</title>`
	`  <taskbody>`
	`  <steps>`
	`  <step>`
	`  <cmd>Configure the server.</cmd>`
	`  <substeps>`
	`  <substep>`
	`  <cmd>Select the authentication mode.</cmd>`
	`  <choices>`
	`  <choice>Local Authentication</choice>`
	`  <choice>LDAP Authentication</choice>`
	`  <choice>SAML Authentication</choice>`
	`  </choices>`
	`  </substep>`
	`  </substeps>`
	`  </step>`
	`  <step>`
	`  <cmd>Restart the service.</cmd>`
	`  </step>`
	`  </steps>`
	`  </taskbody>`
	`</task>`

Result: Level 0 maps to `<steps><step><cmd>` and Level 1 maps to `<substeps><substep><cmd>`, same as a standard multi-level procedure. Level 2 `<ul>` items are mapped them to `<choices><choice>` instead of `<info><ol>`.

Scenario 12 — Admonition Between Numbered List in a Procedure

Consider a procedure where the author has inserted an admonition between two numbered steps to

Consider a procedure where the author has inserted an admonition between two numbered steps to call out a safety or precautionary detail relevant to the step just completed.

Table 29. Admonition between steps in procedure

HTML Input	DITA Output

`<h2 id="update-firmware">Update Firmware</h2>`
`<ol>`
`  <li>`
`  Download the firmware file.`
`  <div class="warning">`
`  Do not disconnect power during the update.`
`  </div>`
`  </li>`
`  <li>Run the update utility.</li>`
`</ol>`
	`<task id="update_firmware">`
	`  <title>Update Firmware</title>`
	`  <taskbody>`
	`  <steps>`
	`  <step>`
	`  <cmd>Download the firmware file.</cmd>`
	`  <info>`
	`  <note type="warning">`
	`  <p>Do not disconnect power during the update.</p>`
	`  </note>`
	`  </info>`
	`  </step>`
	`  <step>`
	`  <cmd>Run the update utility.</cmd>`
	`  </step>`
	`  </steps>`
	`  </taskbody>`
	`</task>`

Result: The converter treats the admonition `<div>` as supporting information tied to the preceding step and nests it inside `<info><note>` rather than splitting the procedure into two separate `<steps>` blocks. The `class` attribute (`warning, note`, etc.) determines the type attribute of the generated `<note>`.

Scenario 13 — Admonition Between Numbered List in a Workflow

Consider a workflow where the author has inserted an admonition between two numbered items to

Consider a workflow where the author has inserted an admonition between two numbered items to call out supplementary or cautionary information relevant to the stage just described.

Table 30. Admonition between stages in workflow

HTML Input	DITA Output

`<h2 id="release-workflow">Release Workflow</h2>`
`<ol>`
`  <li>`
`  Merge the release branch.`
`  <div class="note">`
`  Merging triggers an automatic build in the CI pipeline.`
`  </div>`
`  </li>`
`  <li>Tag the release.</li>`
`</ol>`
	`<concept id="release_workflow">`
	`  <title>Release Workflow</title>`
	`  <conbody>`
	`  <ol>`
	`  <li>`
	`  Merge the release branch.`
	`  <note>`
	`  <p>Merging triggers an automatic build in the CI pipeline.</p>`
	`  </note>`
	`  </li>`
	`  <li>`
	`  Tag the release.`
	`  </li>`
	`  </ol>`
	`  </conbody>`
	`</concept>`

Result: The admonition is nested inside the preceding `<li>` as a `<note>` element rather than splitting the `<ol>` into two independent lists.

Scenario 14 — Paragraph Between Numbered List in a Procedure

Consider a procedure where the author has inserted a <p> element between two numbered steps

Consider a procedure where the author has inserted a `<p>` element between two numbered steps to add explanatory text before continuing the numbered list.

Table 31. Paragraph between steps in procedure

HTML Input	DITA Output
`<h2 id="configure-backup">Configure Backup</h2>` `<ol>`	`<task id="configure_backup">` `  <title>Configure Backup</title>`

Table 31. Paragraph between steps in procedure (continued)

HTML Input	DITA Output
 Open Backup Settings. The backup location must have at least 10 GB of free space. - Select the backup destination. 	<taskbody> <steps> <step> <cmd>Open Backup Settings.</cmd> <info> The backup location must have at least 10 GB of free space. </info> </step> <step> <cmd>Select the backup destination.</cmd> </step> </steps> </taskbody> </task>

Result: The `<p>` element between two numbered steps is nested inside the preceding step's `<info>` element and doesn't break the procedure into separate `<steps>` blocks.

Scenario 15 — Paragraph Between Numbered List in a Workflow

Consider a workflow where the author has inserted a element between two numbered items

Consider a workflow where the author has inserted a `<p>` element between two numbered items to add explanatory text before continuing the numbered list.

Table 32. Paragraph between stages in workflow

HTML Input	DITA Output
<h2 id="approval-workflow">Approval Workflow</h2> Author submits the draft. Submissions are queued and reviewed in the order received.	<concept id="approval_workflow"> <title>Approval Workflow</title> <conbody> Author submits the draft.

Table 32. Paragraph between stages in workflow (continued)

HTML Input	DITA Output
  - Reviewer approves or rejects the draft. 	Submissions are queued and reviewed in the order received. Reviewer approves or rejects the draft. </conbody> </concept>

Result: The `<p>` element is nested inside the preceding `<li>` as a `<p>` element and does not split the `<ol>` into two independent lists.

Scenario 16 — Block Image Between Numbered List in a Procedure

Consider a numbered procedure where the author has inserted a full-width screenshot between two steps

Consider a numbered procedure where the author has inserted a full-width screenshot between two steps — for example, to illustrate what the screen looks like after completing step one.

Table 33. Block image between steps, procedure

HTML Input	DITA Output
<h2 id="open-application">Open the Application</h2> Open the application. <figure> <figcaption> The application's main window </figcaption> </figure> 	<task id="open_application"> <title>Open the Application</title> <taskbody> <steps> <step> <cmd>Open the application.</cmd> <info> <fig> <title>The application main window</title> <image href="media/screenshot.png"> <alt>Application main window</alt>

Table 33. Block image between steps, procedure (continued)

HTML Input	DITA Output
 Click File > Open . 	</image> </fig> </info> </step> <step> <cmd> Click <menucascade> <uicontrol>File</uicontrol> <uicontrol>Open</uicontrol> </menucascade>. </cmd> </step> </steps> </taskbody> </task>

Result: The converter treats the `<figure>` element nested inside a `<li>` as supporting content for that step and moves it into the preceding `<step><info>` element, so that the task remains a single continuous `<steps>` structure.

Scenario 17 — Block Image Between Numbered List in a Workflow

Consider a numbered list where a full-width screenshot is placed between two steps in a

Consider a numbered list where a full-width screenshot is placed between two steps in a workflow to illustrate the application state after completing step one.

Table 34. Block image between stages, workflow

HTML Input	DITA Output
<h2 id="deployment-workflow">Deployment Workflow</h2> The following workflow shows the deployment sequence. Development Environment	<concept id="deployment_workflow"> <title>Deployment Workflow</title> <conbody>

Table 34. Block image between stages, workflow (continued)

HTML Input	DITA Output
 <figure> <figcaption>Deployment workflow transition</figcaption> </figure> - Staging Environment - UAT Environment - Production Environment 	The following workflow shows the deployment sequence. Development Environment <fig> <title>Deployment workflow transition</title> <image href="media/deployment_workflow.png"> <alt>Deployment workflow transition</alt> </image> </fig> - Staging Environment - UAT Environment - Production Environment </conbody> </concept>

Result: The image is in alignment with the numbered list, so the image is inserted, inside a `<fig>` element.

Scenario 18 — Table Between Numbered List in a Workflow

Consider a workflow represented as a numbered list where the author inserts a reference table between lifecycle stages.

Table 35. Table between stages in workflow

HTML Input	DITA Output
<h2 id="deployment-lifecycle"> Deployment Environment Lifecycle	<concept id="deployment_environment_lifecycle"> <title>Deployment Environment Lifecycle</title>

Table 35. Table between stages in workflow (continued)

HTML Input	DITA Output
</h2> The following workflow shows the deployment lifecycle. - Development Environment <table> <caption>Environment responsibilities</caption> <tr> <th>Environment</th> <th>Purpose</th> </tr> <tr> <td>Development</td> <td>Feature implementation</td> </tr> <tr> <td>Staging</td> <td>Pre-production validation</td> </tr> </table> - Staging Environment - Production Environment 	<conbody> The following workflow shows the deployment lifecycle. Development Environment <table> <title>Environment responsibilities</title> <tgroup cols="2"> <colspec colname="c1"/> <colspec colname="c2"/> <thead> <row> <entry>Environment</entry> <entry>Purpose</entry> </row> </thead> <tbody> <row> <entry>Development</entry> <entry>Feature implementation</entry> </row> <row> <entry>Staging</entry> <entry>Pre-production validation</entry> </row> </tbody> </tgroup> </table> - Staging Environment - Production Environment

Table 35. Table between stages in workflow (continued)

HTML Input	DITA Output
	</conbody> </concept>

Result: The converter treats the inserted table as content inside the numbered workflow structure — the table is nested inside the preceding `<li>` and the list stays as a single, unsplit `<ol>`.

Scenario 19 — Table Between Numbered List in a Procedure

Consider a procedure where the author inserts a configuration table between two numbered steps as plain reference information.

Table 36. Table between steps in procedure

HTML Input	DITA Output
<h2 id="configure-server">Configure the Server</h2> Open the server configuration file. <table> <caption>Server parameters</caption> <tr> <th>Parameter</th> <th>Value</th> </tr> <tr> <td>Port</td> <td>8080</td> </tr> <tr> <td>Timeout</td> <td>60</td> </tr> </table> 	<task id="configure_server"> <title>Configure the Server</title> <taskbody> <steps> <step> <cmd>Open the server configuration file.</cmd> <info> <table> <title>Server parameters</title> <tgroup cols="2"> <colspec colname="c1"/> <colspec colname="c2"/> <thead> <row> <entry>Parameter</entry> <entry>Value</entry> </row> </thead> <tbody> <row>

Table 36. Table between steps in procedure (continued)

HTML Input	DITA Output
Save the configuration file. 	<entry>Port</entry> <entry>8080</entry> </row> <row> <entry>Timeout</entry> <entry>60</entry> </row> </tbody> </tgroup> </table> </info> </step> <step> <cmd>Save the configuration file.</cmd> </step> </steps> </taskbody> </task>

Result: The table is mapped to `<choicetable>` because it presents options that the user must choose from while performing the step, rather than a plain reference table.

Scenario 20 — Bullet List between Numbered List in a Procedure

Consider a procedure where the author inserts a bulleted list of prerequisite or reference items

Consider a procedure where the author inserts a bulleted list of prerequisite or reference items between two numbered steps to provide supporting information before the next action.

Table 37. Bullet list between steps in procedure

HTML Input	DITA Output
<h2 id="prepare-the-environment">Prepare the Environment</h2> 	<task id="prepare_the_environment"> <title>Prepare the Environment</title> <taskbody>

Table 37. Bullet list between steps in procedure (continued)

HTML Input	DITA Output
 Verify the following prerequisites are met. - Administrator access is available. - The installer has been downloaded. - Run the installer. 	<steps> <step> <cmd>Verify the following prerequisites are met.</cmd> <info> - Administrator access is available. - The installer has been downloaded. </info> </step> <step> <cmd>Run the installer.</cmd> </step> </steps> </taskbody> </task>

Result: The converter associates the bullet list with the preceding procedural step and places all supporting content inside `<step><info>`.

Scenario 21 — Nested Bullet List Between Numbered List in a Procedure

Consider a procedure where a step's supporting bullet list itself contains a second, deeper level

Consider a procedure where a step's supporting bullet list itself contains a second, deeper level of nested bullets — for example, a general prerequisite item followed by a sub-list of specific requirements under one of its bullets.

Table 38. Nested bullet list between steps

HTML Input	DITA Output
<h2 id="prepare-the-environment">Prepare the Environment</h2> 	<task id="prepare_the_environment"> <title>Prepare the Environment</title> <taskbody>

Table 38. Nested bullet list between steps (continued)

HTML Input	DITA Output
 Verify the following prerequisites are met. - Administrator access is available. Network requirements: - Static IP address configured. - Port 443 open for outbound traffic. - Run the installer. 	<steps> <step> <cmd>Verify the following prerequisites are met.</cmd> <info> - Administrator access is available. - Network requirements: - Static IP address configured. - Port 443 open for outbound traffic. </info> </step> <step> <cmd>Run the installer.</cmd> </step> </steps> </taskbody> </task>

Result: The converter preserves both bullet levels as supporting information rather than promoting either level to a procedural step. The mapping becomes:

- Parent numbered item → `<step><cmd>`
- First-level bullets → `<info><ul>`
- Second-level bullets → a nested `<ul>` inside the enclosing `<li>`

Similarly, consider a procedure where one of the selectable options itself carries a short explanatory sub-list — for example, clarifying what additional input or configuration a specific option requires, rather than introducing separate procedural steps or additional top-level choices.

Table 39. Sub-list for a selectable option

HTML input	DITA output
<h2 id="select-installation-type">Select Installation Type</h2> Choose the installation type. - Typical Installation - Custom Installation <!-- bullet nested inside a choice item --> - Requires manual selection of components. - Requires a target install path. - Start the installation. 	<task id="select_installation_type"> <title>Select Installation Type</title> <taskbody> <steps> <step> <cmd>Choose the installation type.</cmd> <choices> <!-- level 1 --> <choice>Typical Installation</choice> <choice>Custom Installation <!-- level 2, nested inside the level-1 choice --> - Requires manual selection of components. - Requires a target install path. </choice> </choices> </step> <step> <cmd>Start the installation.</cmd> </step> </steps> </taskbody> </task>

Result: The converter preserves the outer bullet list as `<choices>`/`<choice>` because the items represent mutually exclusive, user-selectable options rather than plain descriptive bullets. The sub-bullets under **Custom Installation** are retained as a nested `<ul>` inside that specific `<choice>`, since they clarify a condition of that one option rather than introducing separate steps or additional top-level choices. The mapping becomes:

- Parent numbered item → `<step><cmd>`
- First-level bullets → `<choices><choice>`
- The second-level `<ul>` stays nested inside its parent `<choice>`, scoped only to the option it clarifies

Unordered Lists

This section covers Unordered Lists — items presented as a co-equal collection where position does not affect meaning — and seven scenarios: consecutive bullet items, two-level nested bullet lists, three-level nested bullet lists, a block image between bullet items, a paragraph between bullet items, a table inside a list item, and an admonition between bullet items. The converter maps each unordered list to `<ul><li>` in concept and reference topics, or `<choices>` in task topics, based on the heading classification and content context.

An **unordered list** presents a collection of co-equal items where position does not affect meaning. The unordered list is typically used for prerequisites, supported platforms, product features, available options, feature list, and even terms and definition.

For the reader, a bulleted list communicates that the items are parallel and independent — none depends on another, and the set is complete as presented. The reader does not need to track position or sequence, only membership in the set.

The list level is controlled by the Increase and Decrease Indent buttons, determines nesting depth: level 1 is the top level, level 2 is the first nested level, level 3 is the second, and so on. Sometimes they are also defined by the tab alignment.



Note:
The list level assigned by list controls is what drives the nesting structure in the DITA output.

In well-structured HTML, unordered lists are created using native `<ul>` and `<li>` elements. Nested unordered lists represent sub-groupings within a parent item, while introductory paragraphs, images, tables, and notes provide supporting information without altering the non-sequential relationship between items.

The converter evaluates both the HTML list structure and the classified DITA topic type when processing an unordered list. A `<ul>` immediately preceding an `<ol>` within the same topic typically signals prerequisite or conditional information rather than an independent list. Within Concept, Reference, and Generic topics, an HTML `<ul>` is converted to a DITA `<ul>`. Within a Task topic, a `<ul>` positioned immediately before the procedural `<ol>` is converted into `<prereq>`, with each `<li>` mapped to a `<p>` or `<li>`

inside that section. The surrounding topic context determines whether the list remains a DITA `<ul>` or is transformed into another DITA structure, such as `<prereq>`.

Scenario 1 — Consecutive bullet items

Consider a section titled "Supported Platforms" that contains multiple consecutive <li> items representing related information,

Consider a section titled "Supported Platforms" that contains multiple consecutive `<li>` items representing related information, where sequence does not matter.

Table 40. Consecutive bullet items

HTML Input	DITA Output

`<h2 id="supported-platforms">Supported Platforms</h2>`
`<p>`
`  The application supports the following operating`
` systems.`
`</p>`
`<ul>`
`  <li>Windows 10 and later</li>`
`  <li>macOS 12 and later</li>`
`  <li>Ubuntu 22.04 LTS</li>`
`</ul>`
	`<concept id="supported_platforms">`
	`  <title>Supported Platforms</title>`
	`  <conbody>`
	`  <p>`
	`  The application supports the following operating`
	` systems.`
	`  </p>`
	`  <ul>`
	`  <li>Windows 10 and later</li>`
	`  <li>macOS 12 and later</li>`
	`  <li>Ubuntu 22.04 LTS</li>`
	`  </ul>`
	`  </conbody>`
	`</concept>`

Result: Each `<li>` element is mapped to a DITA `<li>` element inside a `<ul>` inside `<conbody>`. The introductory `<p>` element is preserved before the list.

Now similarly, consider a task topic where a `<ul>` list of prerequisites appears before an `<ol>` list of procedural steps.

Table 41. Prerequisites list before procedural steps

HTML Input	DITA Output
<h2 id="install-the-application"> Install the Application </h2> Ensure the following prerequisites are met before installation. - Python 3.10 or later 4 GB RAM minimum - sudo access - Download the installer. 	<task id="install_the_application"> <title>Install the Application</title> <taskbody> <prereq> Ensure the following prerequisites are met before installation. - Python 3.10 or later 4 GB RAM minimum - sudo access </prereq> <steps> <step> <cmd>Download the installer.</cmd> </step> </steps> </taskbody> </task>

Result: The converter interprets the `<ul>` list appearing before the first `<ol>` as a prerequisites block and maps it to `<ul>` inside `<prereq>`. The `<ol>` list that follows is mapped to `<steps><step><cmd>`.

Scenario 2 — Two-level nested bullet list

Consider a <ul> list with a second tier of nested <ul> sub-items beneath selected parent

Consider a `<ul>` list with a second tier of nested `<ul>` sub-items beneath selected parent `<li>` items.

Table 42. Two-level nested bullet list

HTML Input	DITA Output
<h2 id="system-requirements">System Requirements</h2> Supported platforms - Windows 10 and later - macOS 12 and later Prerequisites 8 GB RAM minimum 20 GB free disk space 	<concept id="system_requirements"> <title>System Requirements</title> <conbody> Supported platforms - Windows 10 and later - macOS 12 and later Prerequisites 8 GB RAM minimum 20 GB free disk space </conbody> </concept>

Result: Nested `<ul>` elements are preserved when native HTML list nesting is applied correctly. Each nested `<ul>` is placed inside its parent `<li>` element in the DITA output.

Note:
Never simulate indentation using ` ` characters or CSS padding inside a `<p>` element — this does not create a nested list structure and the converter does not recognize it as such.

Scenario 3 — Three-level deep nested bullet list

Consider a <ul> list nested three levels deep — grandchild <li> items indented beneath child

Consider a `<ul>` list nested three levels deep — grandchild `<li>` items indented beneath child `<li>`; items using nested `<ul>` elements.

Table 43. Three-level deep nested bullet list

HTML Input	DITA Output

`<h2 id="infrastructure-types">Infrastructure Types</h2>`
`<ul>`
`  <li>`
`  Hardware`
`  <ul>`
`  <li>`
`  Servers`
`  <ul>`
`  <li>Rack-mounted</li>`
`  <li>Tower</li>`
`  </ul>`
`  </li>`
`  </ul>`
`  </li>`
`</ul>`
	`<concept id="infrastructure_types">`
	`  <title>Infrastructure Types</title>`
	`  <conbody>`
	`  <ul>`
	`  <li>`
	`  Hardware`
	`  <ul>`
	`  <li>`
	`  Servers`
	`  <ul>`
	`  <li>Rack-mounted</li>`
	`  <li>Tower</li>`
	`  </ul>`
	`  </li>`
	`  </ul>`
	`  </li>`
	`  </ul>`
	`  </conbody>`
	`</concept>`

Result: The converter produces valid DITA for three-level nesting. Each level of nested `<ul>` is preserved inside its parent `<li>` element.

Note:
Avoid nesting beyond three levels. Restructure deeply nested content using separate topics or a table instead.

Scenario 4 — Block image placed between bullet list items

Consider a section where the author inserts a block image between bullet items to visually

Consider a section where the author inserts a block image between bullet items to visually illustrate one of the items.

Table 44. Block image between bullet items

HTML Input	DITA Output

`<h2 id="deployment-models">`
`  Supported Deployment Models`
`</h2>`
`<ul>`
`  <li>On-premises deployment</li>`
`</ul>`
`<figure>`
`  <img src="media/deployment_architecture.png" alt="Deployment`
` architecture overview"/>`
`  <figcaption>`
`    Deployment architecture overview`
`  </figcaption>`
`</figure>`
`<ul>`
`  <li>Cloud deployment</li>`
`  <li>Hybrid deployment</li>`
`</ul>`
	`<concept id="supported_deployment_models">`
	`  <title>Supported Deployment Models</title>`
	`  <conbody>`
	`    <ul>`
	`      <li>`
	`        On-premises deployment`
	`        <fig>`
	`          <title>Deployment architecture`
	` overview</title>`
	`          <image`
	` href="media/deployment_architecture.png">`
	`            <alt>Deployment architecture`
	` overview</alt>`
	`          </image>`
	`        </fig>`
	`      </li>`
	`      <li>Cloud deployment</li>`
	`      <li>Hybrid deployment</li>`
	`    </ul>`
	`  </conbody>`
	`</concept>`

Result: The converter treats the two adjacent `<ul>` elements separated only by a `<figure>` are merged into one, with the image nested inside the last `<li>`of the first segment.

Scenario 5 — Paragraph Between a Bullet list item

Consider a bulleted list where the author has closed the <ul> element after a list

Consider a bulleted list where the author has closed the `<ul>` element after a list item, inserted a `<p>` element containing explanatory text, and then opened a new `<ul>` element to continue the list. Because

the paragraph sits outside both list elements, it has no `<li>` markup, even though it appears visually between two bullets.

Table 45. Paragraph between bullet items

HTML Input	DITA Output
`<h2>Backup Guidelines</h2><ul><li>Back up the configuration file.</li></ul><p>Make sure the backup is stored outside the install directory.</p><ul><li>Restart the service to apply the configuration.</li></ul>`	`<concept id="backup_guidelines"><title>Backup Guidelines</title><conbody><ul><li>Back up the configuration file.<p>Make sure the backup is stored outside the install directory.</p></li><li>Restart the service to apply the configuration.</li></ul></conbody></concept>`

Result: The result depends on how the AI Agent interprets the split. If it infers continuation, the paragraph is nested inside the preceding `<li>` and the two `<ul>` elements are merged into a single DITA `<ul>`; otherwise, the paragraph and each `<ul>` are kept as separate, adjacent elements in the output.

Scenario 6 — Table embedded inside a list item

Consider a `<ul>` list where the author has inserted a `<table>` element directly inside one of the `<li>` items — for example, to present a small reference table as part of a bullet point.

Table 46. Table embedded in list item

HTML Input	DITA Output
`<ul><li>`	`<conbody><ul>`

Table 46. Table embedded in list item (continued)

HTML Input	DITA Output
The following options are available: <table> <tr> <th>Option</th> <th>Default</th> </tr> <tr> <td>Retry</td> <td>3 times</td> </tr> <tr> <td>Timeout</td> <td>30 sec</td> </tr> </table> 	 The following options are available: <simpletable> <sthead> <stentry>Option</stentry> <stentry>Default</stentry> </sthead> <strow> <stentry>Retry</stentry> <stentry>3 times</stentry> </strow> <strow> <stentry>Timeout</stentry> <stentry>30 sec</stentry> </strow> </simpletable> </conbody>

Result: When a `<table>` element is correctly nested inside an `<li>` element in the HTML source, the converter maps it to a `<simpletable>` inside the corresponding DITA `<li>` element. Always verify that the table-inside-list structure is preserved correctly in the HTML source before conversion.

Scenario 7 — Admonition between Bullet List

Consider a bulleted list where the author has inserted an admonition between two bullet items

Consider a bulleted list where the author has inserted an admonition between two bullet items to call out a precaution or clarification relevant to the item just listed.

Table 47. Admonition between bullet items

HTML Input	DITA Output
	<concept id="supported_deployment_models">

Table 47. Admonition between bullet items (continued)

HTML Input	DITA Output

`<h2 id="deployment-models">Supported Deployment Models</h2>`
`<ul>`
`<li>`
`On-premises deployment`
`<div class="warning">`
`Requires dedicated IT staff to maintain hardware.`
`</div>`
`</li>`
`<li>Cloud deployment</li>`
`<li>Hybrid deployment</li>`
`</ul>`
	`<title>Supported Deployment Models</title>`
	`<conbody>`
	`<ul>`
	`<li>`
	`On-premises deployment`
	`<note type="warning">`
	`<p>Requires dedicated IT staff to maintain hardware.</p>`
	`</note>`
	`</li>`
	`<li>Cloud deployment</li>`
	`<li>Hybrid deployment</li>`
	`</ul>`
	`</conbody>`
	`</concept>`

Result: The admonition is nested inside the preceding `<li>` as a `<note>` element, and the `<ul>` remains a single continuous list.

Term List (Definition List)

This section covers Term Lists (Definition Lists) in HTML — structures that associate a term with its corresponding definition or description, emphasizing term-description relationships rather than value comparison across columns — and two scenarios: a native HTML `<dl>` element with `<dt>` and `<dd>` pairs, and a `<ul>` list following the bold-label pattern where each `<li>` begins with a `<strong>` term followed by a separator and definition text. The converter maps each term list to `<dl><dlentry><dt><dd>` in concept and reference topics, or nests it inside `<step><info>` as supporting information in task topics, based on the heading classification and content context.

A **term list**, also known as a definition list, is an HTML structure used to associate terms with their corresponding definitions or descriptions. It is commonly used to present technical terminology, configuration options, error codes, connection modes, and other reference information, with each term followed by one or more explanatory statements. Unlike a table, a definition list emphasizes the relationship between a term and its description rather than comparing values across multiple columns.

Term lists improve readability by allowing related definitions to be scanned without interrupting the flow of the surrounding content. Publishing systems, accessibility technologies, and content management systems rely on correctly structured definition lists to preserve the relationship between each term and its description across different output formats.

In well-structured HTML, term lists are authored either as native `<dl>` elements containing `<dt>` and `<dd>` pairs or as unordered lists where each `<li>` begins with a `<strong>` term followed by a separator and its corresponding definition. The native `<dl>` structure provides the clearest semantic representation, while the bold-label pattern offers an alternative that the converter can also recognize.

The converter detects both supported HTML patterns when generating DITA output. Native `<dl>` elements map directly to DITA `<dl>`, while recognized `<ul>` lists that follow the bold-label pattern are converted to the same DITA definition list structure. Each term becomes a DITA `<dt>`, each definition becomes a `<dd>`, and related pairs are grouped within a `<dlentry>`.



Important:
Use native HTML `<dl>`, `<dt>`, and `<dd>` elements wherever possible to represent term-definition relationships. Although the converter recognizes supported `<ul>` patterns, native definition lists provide the clearest semantic structure and produce the most reliable conversion results.

The following table summarizes the DITA elements used to represent term lists during HTML-to-DITA conversion:

Table 48. Definition list elements and descriptions

DITA Element	Description
`<dl>`	Definition list container.
`<dlentry>`	Groups a single term–definition pair. Contains exactly one `<dt>` and one or more `<dd>` elements.
`<dt>`	Definition term. Maps from the `<dt>` element or the `<strong>` run at the start of a `<li>` item.
`<dd>`	Definition description. Maps from the `<dd>` element or the non-bold text that follows the term in the same `<li>` item.

Definition lists are commonly used inside concept and reference topics. When a definition list appears in a task topic, it is typically placed inside `<step><info>` as supporting information.

Scenario 1 — Native <dl> element

Scenario 1 — Native <dl> element Consider a concept section titled "Connection Modes" where the

Consider a concept section titled "Connection Modes" where the author has used a native HTML `<dl>` element with `<dt>` and `<dd>` pairs.

Table 49. Native dl/dt/dd list element

HTML Input	DITA Output
<h2 id="connection-modes">Connection Modes</h2> The device supports the following connection modes. <dl> <dt>Wired</dt> <dd> Uses an Ethernet cable to connect the device directly to the network switch. </dd> <dt>Wireless</dt> <dd> Connects the device using the built-in Wi-Fi adapter. </dd> <dt>Bluetooth</dt> <dd> Pairs the device with a host computer for short-range data transfer. </dd> </dl>	<concept id="connection_modes"> <title>Connection Modes</title> <conbody> The device supports the following connection modes. <dl> <dlentry> <dt>Wired</dt> <dd> Uses an Ethernet cable to connect the device directly to the network switch. </dd> </dlentry> <dlentry> <dt>Wireless</dt> <dd> Connects the device using the built-in Wi-Fi adapter. </dd> </dlentry> <dlentry> <dt>Bluetooth</dt> <dd> Pairs the device with a host computer for short-range data transfer. </dd> </dlentry> </dl>

Table 49. Native dl/dt/dd list element (continued)

HTML Input	DITA Output
	</conbody> </concept>

Result: The native HTML `<dl>` structure maps directly to DITA. The mapping is: `<dl>` → `<dl>`; each `<dt>` → `<dlentry><dt>`; each `<dd>` → `<dlentry><dd><p>`.

Scenario 2 — list with bold term and separator pattern

Scenario 2 — list with bold term and separator pattern Consider a concept section

Consider a concept section titled "Connection Modes" where the author has used a `<ul>` list where each `<li>` begins with a `<strong>` term followed by a — separator and definition text.

Table 50. Bold term with separator pattern

HTML Input	DITA Output
<h2 id="connection-modes">Connection Modes</h2> The device supports the following connection modes. Wired — Uses an Ethernet cable to connect the device directly to the network switch. Wireless — Connects the device using the built-in Wi-Fi adapter. Bluetooth — Pairs the device with a host computer for short-range data transfer. 	<concept id="connection_modes"> <title>Connection Modes</title> <conbody> The device supports the following connection modes. <dl> <dlentry> <dt>Wired</dt> <dd> Uses an Ethernet cable to connect the device directly to the network switch. </dd> </dlentry> <dlentry> <dt>Wireless</dt>

Table 50. Bold term with separator pattern (continued)

HTML Input	DITA Output
	<dd> Connects the device using the built-in Wi-Fi adapter. </dd> </dlentry> <dlentry> <dt>Bluetooth</dt> <dd> Pairs the device with a host computer for short-range data transfer. </dd> </dlentry> </dl> </conbody> </concept>

Result: The converter detects the bold-term pattern and maps the `<ul>` to a DITA `<dl>`. Each `<li>` becomes a `<dlentry>`, the `<strong>` text maps to `<dt>`, and the remaining text maps to `<dd>`. The separator character is stripped.

Chapter 10. Images and Graphics

This section covers Images and Graphics in HTML — the primary means of communicating spatial relationships, component locations, interface states, and process flows that prose alone cannot convey — and twelve scenarios: image sizing thresholds, inline and block placement, images with captions, side-by-side images, images inside list items, linked versus embedded images, images inside text boxes, and alt text application. The converter maps each image to a DITA `<image>` element with a required `<alt>` child; block images are wrapped in `<fig>`, and a Caption-styled paragraph maps to the `<fig>` `<title>` element.

Images and graphics are the primary means of communicating spatial relationships, component locations, interface states, system architectures, and process flows that text alone cannot convey efficiently. In technical publications, the image or graphic and its surrounding context carry equal structural weight.

The key differences between images and graphics are outlined below:

- An image is a raster file — a grid of pixels captured or rendered at a fixed resolution. Images are resolution dependent, when scaled up beyond their original size, they lose sharpness. For example, screenshots, photographs, and scanned content (PNG, JPEG, BMP).
- A graphic is a vector-based or constructed visual — built from mathematical shapes, lines, curves, and text rather than pixels. Graphics are resolution independent, when scaled up beyond their size does not lose its quality. For example, diagrams, flowcharts, and schematic illustrations (SVG, EMF, Visio, PDF).

> **Tip:**
> Embed every image directly in the document body, apply meaningful alt text for accessibility, and add a caption to every figure that requires a title in the published output.

In well-structured HTML, images are referenced through valid src paths and positioned according to their intended role within the document. Images placed between block-level elements typically represent standalone figures, while images embedded within a paragraph support the surrounding text. When a `<figure>` element includes a `<figcaption>`, the caption provides a descriptive title for the generated figure.

The converter evaluates each `<img>` element together with its surrounding HTML context to determine the appropriate DITA structure. Standalone images are converted to `<fig>` elements containing `<image>`, inline images remain inline within the surrounding paragraph, and `<figure>` elements with `<figcaption>` generate titled DITA `<fig>` elements. During conversion, image files are extracted, converted to PNG format, stored in the media/ subfolder of the output directory, and referenced from the generated DITA topics using relative paths.

Warning:
If an image src path cannot be resolved, the converter either generates an unresolved `<image>` reference or omits the image from the DITA output. Verify that all image files are present and correctly referenced in the HTML package before conversion.

All images referenced in the HTML document are converted to .png format in the DITA output and saved to the media/ subfolder of the output directory.

The following table lists the image positions and their corresponding DITA output:

Table 51. Image position mapped to DITA

Image Position in HTML	DITA Output Element

`<img>` element between block-level elements	`<fig>`
	`  <image href="media/diagram.png"/>`
	`</fig>`

`<img>` element inline within a sentence inside `<p>`	`<p>`
	`  Press the`
	`  <image href="media/ok_btn.png"`
	` placement="inline"/>`
	`  button.`
	`</p>`

`<img>` element followed by a `<figcaption>` inside `<figure>`	`<fig>`
	`  <title>Ink tank and cap locations</title>`
	`  <image href="media/ink_tank.png"/>`
	`</fig>`

`<img>` element with no accessible src	`Unresolved <image> reference or omitted from` ` output.`

Scenario 1 — Image Size is Smaller than 50 Pixels

Consider a paragraph that contains a small inline image of a "next button" whose width

Consider a paragraph that contains a small inline image of a "next button" whose width and height attributes are both smaller than 50 pixels.

Table 52. Image smaller than 50 pixels

HTML Input	DITA Output

`<p>`
`  Click the <strong>Next</strong> button`
`  <img src="media/next_arrow.png" alt="Next arrow"`
` width="20" height="24"/>`
`  to continue to the next page.`
`</p>`
	`<p>`
	`  Click the <uicontrol>Next</uicontrol> button`
	`  <image href="media/next_arrow.png" placement="inline"`
	` width="5" height="6">`
	`  <alt>Next arrow</alt>`
	`  </image>`
	`  to continue to the next page.`
	`</p>`

Result: If the `<img>` element's width and height are both smaller than **50 × 50 pixels**, the converter generates an `<image/>` element with the placement attribute as inline. The converter does not wrap the image in a `<fig>` tag.

Scenario 2 — Image Size is Larger than 50 Pixels

Consider a paragraph that contains a medium-sized image of a "dialog box" whose width and

Consider a paragraph that contains a medium-sized image of a "dialog box" whose width and height attributes are both larger than 50 pixels.

Table 53. Image larger than 50 pixels

HTML Input	DITA Output

`<p>`
`  The following dialog box displays the available network`
` settings:`
	`<p>`
	`  The following dialog box displays the available`
	` network settings:`

Table 53. Image larger than 50 pixels (continued)

HTML Input	DITA Output
<pre><code>\n </code></pre>	<pre><code><image href="media/network_settings_dialog.png" placement="break" width="4" height="3">\n<alt>Network settings dialog</alt>\n</image>\n </code></pre>

Result: If the `<img>` element's width and height are equal to or larger than **50 × 50 pixels**, the converter generates an `<image/>` element with the placement attribute as break. The converter does not wrap the image in a `<fig>` tag.

Scenario 3 — Image Size is Larger than 100 Pixels

Consider a paragraph that introduces a large architecture diagram whose width and height attributes are

Consider a paragraph that introduces a large architecture diagram whose width and height attributes are both larger than 100 pixels.

Table 54. Image larger than 100 pixels

HTML Input	DITA Output
<pre><code> The following diagram illustrates the overall system architecture. \n</code></pre>	<pre><code> The following diagram illustrates the overall system architecture. \n<fig>\n<title>System Architecture Overview</title>\n<image href="media/system_architecture_overview.png" placement="break" width="13" height="10">\n<alt>System architecture overview</alt>\n</image>\n</fig></code></pre>

Result: If the `<img>` element's width and height are both larger than **100 × 100 pixels**, the converter generates an `<image/>` element with the placement attribute as break and wraps it in a `<fig>` element. The converter also generates the `<title>` element from the image filename. The `<fig>` element is placed outside the preceding `<p>` element so that the image is represented as a standalone figure.

Scenario 4 — Inline image within correct wrap mode

Consider a `<p>` element where an `<img>` element appears inline within the sentence text — for example, to show a button icon as part of an instruction.

Table 55. Inline image within correct wrap mode

HTML Input	DITA Output

`<p>`
`  Press the`
`  <img src="media/ok_btn.png"`
`  alt="OK button"/>`
`  button to confirm.`
`</p>`
	`<p>`
	`  Press the`
	`  <image href="media/ok_btn.png" placement="inline">`
	`  <alt>OK button</alt>`
	`  </image>`
	`  button to confirm.`
	`</p>`

Result: The converter detects the `<img>` element positioned inside a `<p>` element alongside text and maps it to `<image placement="inline">` within the surrounding `<p>` element. The alt attribute is mapped to the `<alt>` element inside `<image>`.

Scenario 5 — Image with caption

Consider an image of the printer's ink tank area wrapped in a <figure> element with

Consider an image of the printer's ink tank area wrapped in a `<figure>` element with a `<figcaption>` element reading "Figure 1: Ink tank and cap locations".

Table 56. Image with caption

HTML Input	DITA Output

`<figure>`
`  <img src="media/ink_tank_area.png" alt="Ink tank and cap`
` locations"/>`
`  <figcaption>`
`  Figure 1: Ink tank and cap locations`
	`<fig>`
	`  <title>Ink tank and cap`
	` locations</title>`
	`  <image href="media/ink_tank_area.png">`
	`  <alt>Ink tank and cap locations</alt>`

Table 56. Image with caption (continued)

HTML Input	DITA Output
</figcaption> </figure>	</image> </fig>

Result: The `<figure>` wrapper is mapped to a DITA `<fig>` element. The `<figcaption>` text is mapped to the `<title>` element inside `<fig>`. Figure number prefixes such as "Figure 1:" are stripped from the `<title>` text during conversion, retaining only the descriptive label.

Note:
Always use the native HTML `<figure>` and `<figcaption>` elements to add captions to images. A `<p>` element placed above or below an `<img>` and used as a visual caption is mapped to a `<p>` element, not a `<title>`.

Scenario 6 — Multiple images placed side by side

Consider a `<p>` element where the author has inserted two or more `<img>` elements next to each other — for example, a set of toolbar icon images.

Table 57. Multiple images placed side-by-side

HTML Input	DITA Output
    	 <image href="media/icon_a.png" placement="inline"> <alt>Icon A</alt> </image> <image href="media/icon_b.png" placement="inline"> <alt>Icon B</alt> </image> <image href="media/icon_c.png" placement="inline"> <alt>Icon C</alt> </image>

Result: Each `<img>` element is mapped to an `<image placement="inline"/>`element inside the same `<p>`. The images appear in document order.

> **Tip:**
> If a specific side-by-side layout is required in the published output, use a `<table>` with images in adjacent `<td>` cells.

Scenario 7 — Image inside a numbered list item

Consider a numbered procedure where the author has placed an <img> element with a <figcaption>

Consider a numbered procedure where the author has placed an `<img>` element with a `<figcaption>` inside a `<figure>` inside a `<li>` element — for example, to show a button or UI element as part of a step.

Table 58. Image inside numbered list item

HTML Input	DITA Output
    Locate the power button. <figure> <figcaption> Power button location </figcaption> </figure> 	<task id="locate_the_power_button"> <title>Locate the Power Button</title> <taskbody> <steps> <step> <cmd>Locate the power button.</cmd> <info> <fig> <title>Power button location</title> <image href="media/power_button.png"> <alt>Power button</alt> </image> </fig> </info> </step> </steps> </taskbody> </task>

Result: The converter maps the `<figure>` and `<figcaption>` inside the `<li>` to a `<fig>` structure. In a task topic, the `<fig>` is placed inside `<step><info>`, not inside `<cmd>`.

Scenario 8 — Image inside a bulleted list item

Consider a section where the author has placed an <img> element inside an <li> element

Consider a section where the author has placed an `<img>` element inside an `<li>` element of an unordered list — for example, to show a status icon as part of a descriptive list.

Table 59. Image inside bulleted list item

HTML Input	DITA Output
Status Indicators The green status icon indicates that the system is running. The red status icon indicates that an error has occurred. 	<concept id="status_indicators"> <title>Status Indicators</title> <conbody> The green status icon <image href="media/green_status_icon.png" placement="inline"> <alt>Green status icon</alt> </image> indicates that the system is running. The red status icon <image href="media/red_status_icon.png" placement="inline"> <alt>Red status icon</alt> </image> indicates that an error has occurred. </conbody> </concept>

Result: The converter maps the `<ul>` to a DITA `<ul>` and each `<li>` to a DITA `<li>`. Inline `<img>` elements inside each `<li>` are preserved as `<image placement="inline"/>` inside the corresponding `<li>` element.

Scenario 9 — Image Linked via External URL Rather than a Local Package Reference

Consider an HTML document where the author has referenced an image using an absolute external

Consider an HTML document where the author has referenced an image using an absolute external URL (for example, hosted on a separate website or CDN) instead of a relative path to a file included in the HTML package. Because the image file is not part of the submitted package, the converter cannot extract it.

Table 60. Image linked via external URL

HTML Input	DITA Output

`<img src="https://external-cdn.example.com/images/diagram.png"`
`alt="Diagram"/>`
	`<!-- Output A: broken reference -->`
	`<image href="media/MISSING_image.png"/>`
	`<!-- Output B: image silently dropped`
	`-->`

Result: Externally hosted images produce either a broken `<image>` reference or are silently dropped, since the converter only extracts image files that are physically included in the submitted HTML/ZIP package. Always include image files locally within the package and reference them with a relative src path before submission.

Scenario 10 — Image Placed Inside a Positioned (Text Box Equivalent)

Consider a document where the author has placed an image inside a `<div>` that is removed from the normal document flow using CSS (for example, `position: absolute` or `position: fixed`) to control its placement on the page, similar to a floating text box.

Result: The converter ignores the CSS positioning and any surrounding content that isn't structurally part of the body flow, but it retains the <img> element placed inside the positioned `<div>`. The extracted image is mapped to a DITA `<image>` element, typically wrapped in a `<fig>` if it appears as a block image.

Warning:
Do not place images inside CSS-positioned `<div>` containers that sit outside the normal document flow. Although the image may be retained, any surrounding positioned content can be lost during conversion.

Scenario 11 — Image with Floating Label Elements Layered on Top

Scenario 11 — Image with Floating Label Elements Layered on Top Consider a document

Consider a document where an `<img>` element is inserted into the document body and multiple `<div>` elements are positioned on top of the image using CSS (for example, `position: absolute`) to create labels, admonitions, or annotations.

Table 61. Floating label elements over image

HTML Input	DITA Output
`  <img src="media/network_topology.png" alt="Network topology"/>  Database Server   Application Server   Load Balancer   `	`<fig> <image href="media/network_topology.png"> <alt>Network topology</alt> </image> </fig> <!-- Positioned/overlaid text labels ("Database Server", "Application Server", "Load Balancer") have no DITA equivalent for absolutely-positioned text and cannot be converted automatically; flagged for manual review. -->`

Result: The converter extracts and preserves the `<img>` element itself because it exists as a valid image reference. However, the CSS-positioned `<div>` labels layered above the image are ignored during conversion.

Tip:
For annotated diagrams, embed labels directly into the image before inserting it into the HTML page.

Scenario 12 — Image with an Alt Text

The alt attribute on an HTML `<img>` element provides a short text description of the image for readers who cannot see it, such as those using screen readers. It is a required attribute for accessibility compliance.

Table 62. Image with alt text

HTML Input	DITA Output

`<img src="media/print_head_diagram.png"`
`alt="Diagram showing the print head path inside the printer"/>`
	`<image href="media/print_head_diagram.png">`
	`<alt>Diagram showing the print head path inside the printer</alt>`
	`</image>`

Result: The converter extracts the `alt` attribute value from the HTML `<img>` element and maps it to the DITA `<alt>` element inside `<image>`, preserving the image description for accessibility and assistive technologies.

Chapter 11. Tables

This section covers the Table style in HTML — the primary structure for presenting information with inherent row-and-column relationships — and ten scenarios: a standard multi-column table, a table with a caption, a table with no header row, merged cells, nested tables, lists and images inside table cells, tables spanning multiple pages, consecutive tables, and a two-column parameter table in a reference topic. The converter reads each table's structure, cell complexity, and topic context to determine whether to map it to `<simpletable>`, a CALS `<table>`, or `<properties>`.

A**Table** is the primary structure for reference and lookup content. It is a structural mechanism to present information that has inherent row-and-column relationships — parameter names and their values, feature names and their states, error codes and their meanings, API fields and their data types.

A table reduces a lookup task from reading a paragraph to scanning a row and column. In DITA-XML, a table is structured data — each cell is a discrete, addressable unit that a transform can reformat for PDF, or online help output, filter by column for conditional publishing, and extract independently for reuse in other deliverables.

A table is an HTML structure used to organize related information into rows and columns. Represented by the native `<table>` element together with `<tr>`, `<th>`, and `<td>`, it presents information that is easier to compare, scan, or reference than the same information written as continuous text. Tables commonly represent configuration parameters, feature comparisons, error codes, API fields, and other structured lookup information.

The converter evaluates each table's structure — merged cells, the presence of a `<caption>`, column count, and the topic type it appears in — to determine the correct DITA output. A simple table maps to `<simpletable>`. A table with merged cells or a `<caption>` maps to a CALS `<table>` with `<tgroup>`. A two-column parameter table inside a Reference topic maps to `<properties>`. The converter relies entirely on the HTML table model, never on visual appearance.

> **Important:**
> Author tables using native `<table>`, `<tr>`, `<th>`, and `<td>` elements. Tables simulated with spacing, `<pre>`, or images are converted as plain text or figures, with no table structure in the output.

The following table summarizes how different HTML table structures are mapped during HTML-to-DITA conversion:

Table 63. HTML table structures mapped to DITA

HTML Table Structure	Topic Type	DITA Output Element
Any `<table>` with a `<caption>` element	Any	CALS `<table>` + `<tgroup>`; `<caption>` → `<title>`
Any `<table>` with merged cells (colspan or rowspan)	Any	CALS `<table>` + `<tgroup>`
2-column `<table>`, no caption, no merge	Reference (API keyword context)	`<properties>` with `<prop-type>`/`<propvalue>`
2-column `<table>`, no caption, no merge	Concept / Task / Generic	`<simpletable>` with `<sthead>`/`<strow>`

Scenario 1 — Standard multi-column table

Consider a status indicator lights table under the heading "Status Indicator Lights". The table has three columns — Light Pattern, Color, and Status — with a header row using `<th>` elements and data rows using `<td>` elements. No merged cells exist.

Table 64. Standard multi-column table

HTML Input

```
<h2>Status Indicator Lights</h2>

<table>
  <tr>
    <th>Light Pattern</th>
    <th>Color</th>
    <th>Status</th>
  </tr>
  <tr>
    <td>Solid</td>
    <td>Green</td>
    <td>Printer is ready.</td>
  </tr>
  <tr>
    <td>Flashing</td>
```

DITA Output

```
<reference id="status_indicator_lights">
  <title>Status Indicator Lights</title>
  <refbody>
    <simpletable>
      <sthead>
        <stentry>Light Pattern</stentry>
        <stentry>Color</stentry>
        <stentry>Status</stentry>
      </sthead>
      <strow>
        <stentry>Solid</stentry>
        <stentry>Green</stentry>
        <stentry>Printer is ready.</stentry>
      </strow>
      <strow>
```

Table 64. Standard multi-column table (continued)

HTML Input	DITA Output
<td>Orange</td> <td>Paper jam detected.</td> </tr> <tr> <td>Solid</td> <td>Red</td> <td>Ink level is critically low.</td> </tr> </table>	<stentry>Flashing</stentry> <stentry>Orange</stentry> <stentry>Paper jam detected.</stentry> </strow> <strow> <stentry>Solid</stentry> <stentry>Red</stentry> <stentry>Ink level is critically low.</stentry> </strow> </simpletable> </refbody> </reference>

Result: `<table>` without merged cells or a `<caption>` maps to `<simpletable>`. `<th>` elements map to `<sthead>` and each `<tr>` maps to `<strow>`.

Scenario 2 — Table with a <caption> element

Consider a configuration parameter table where a `<caption>` element is inside the `<table>` element above the header.

Table 65. Table with caption element

HTML Input	DITA Output
<table> <caption>Server Configuration Parameters</caption> <tr> <th>Parameter</th> <th>Default Value</th> <th>Description</th> </tr> <tr> <td>Port</td> <td>8080</td>	<reference id="server_configuration_parameters"> <title>Server Configuration Parameters</title> <refbody> <table> <title>Server Configuration Parameters</title> <tgroup cols="3"> <colspec colname="c1"/> <colspec colname="c2"/> <colspec colname="c3"/> <thead>

Table 65. Table with caption element (continued)

HTML Input	DITA Output
<td>Server listening port.</td> </tr> <tr> <td>Timeout</td> <td>60</td> <td>Session timeout in seconds.</td> </tr> <tr> <td>Enable SSL</td> <td>True</td> <td>Enables encrypted communication.</td> </tr> </table>	<row> <entry>Parameter</entry> <entry>Default Value</entry> <entry>Description</entry> </row> </thead> <tbody> <row> <entry>Port</entry> <entry>8080</entry> <entry>Server listening port.</entry> </row> <row> <entry>Timeout</entry> <entry>60</entry> <entry>Session timeout in seconds.</entry> </row> <row> <entry>Enable SSL</entry> <entry>True</entry> <entry>Enables encrypted communication.</entry> </row> </tbody> </tgroup> </table> </refbody> </reference>

Result: The converter produces a CALS `<table>` + `<tgroup>` because a `<caption>` element is present. The `<caption>` text is mapped to the DITA `<title>` element inside `<table>`.

Note:
Always use the native HTML `<caption>` element to add captions to tables. Only `<caption>` elements are mapped to DITA `<title>` elements. A `<p>` element placed above a table and used as a visual caption is mapped to a `<p>` element, not a `<title>`.

Scenario 3 — Table with no header row

Consider a table where the author has not used `<th>` elements for the first row — all rows use `<td>` elements only.

Table 66. Table with no header row

HTML Input

```
<table>
  <tr>
    <td>server-01</td>
    <td>192.168.1.10</td>
    <td>Active</td>
  </tr>
  <tr>
    <td>server-02</td>
    <td>192.168.1.11</td>
    <td>Standby</td>
  </tr>
  <tr>
    <td>server-03</td>
    <td>192.168.1.12</td>
    <td>Active</td>
  </tr>
</table>
```

DITA Output

```
<reference id="api_parameters">
  <title>API Parameters</title>
  <refbody>
    <simpletable frame="all" relcolwidth="1* 1* 1*" id="simpletable_yj1_ggk_rjc">
      <sthead>
        <stentry>server-01</stentry>
        <stentry>192.168.1.10</stentry>
        <stentry>Active</stentry>
      </sthead>
      <strow>
        <stentry>server-02</stentry>
        <stentry>192.168.1.11</stentry>
        <stentry>Standby</stentry>
      </strow>
      <strow>
        <stentry>server-03</stentry>
        <stentry>192.168.1.12</stentry>
        <stentry>Active</stentry>
      </strow>
    </simpletable>
  </refbody>
</reference>
```

Result: The converter assumes that every HTML table requires a header row in the DITA output. When no `<th>` elements are present, the converter promotes the first `<tr>` of `<td>` elements into `<sthead>`. Always include a descriptive header row using `<th>` elements in HTML tables.

Scenario 4 — Table with merged cells

Consider a table where cells are merged vertically across rows using the `rowspan` attribute — for example, to associate one server group value with several related rows.

Table 67. Table with merged cells

HTML Input

```
<table>
  <caption>Server Role Mapping</caption>
  <tr>
    <th>Server Group</th>
    <th>Server Name</th>
    <th>Status</th>
  </tr>
  <tr>
    <td rowspan="2">Production</td>
    <td>server-01</td>
    <td>Active</td>
  </tr>
  <tr>
    <td>server-02</td>
    <td>Active</td>
  </tr>
  <tr>
    <td>Backup</td>
    <td>server-03</td>
    <td>Standby</td>
  </tr>
</table>
```

DITA Output

```
<reference id="server_role_mapping">
  <title>Server Role Mapping</title>
  <refbody>
    <table>
      <title>Server Role Mapping</title>
      <tgroup cols="3">
        <colspec colname="c1"/>
        <colspec colname="c2"/>
        <colspec colname="c3"/>
        <thead>
          <row>
            <entry>Server Group</entry>
            <entry>Server Name</entry>
            <entry>Status</entry>
          </row>
        </thead>
        <tbody>
          <row>
            <entry morerows="1">Production</entry>
            <entry>server-01</entry>
            <entry>Active</entry>
          </row>
          <row>
            <entry>server-02</entry>
            <entry>Active</entry>
```

Table 67. Table with merged cells (continued)

HTML Input	DITA Output
	</row> <row> <entry>Backup</entry> <entry>server-03</entry> <entry>Standby</entry> </row> </tbody> </tgroup> </table> </refbody> </reference>

Result: The rowspan=\"2\" maps to morerows=\"1\" on the DITA `<entry>` element. Merged cells always produce CALS `<table>` + `<tgroup>` output.

Warning:
Merged-cell structures with spanning headers, grouped values, or irregular layouts require manual review after conversion. Verify that the generated DITA table accurately reflects the source structure.

Scenario 5 — Nested table (table inside a table cell)

Consider a table where the author has placed a second complete `<table>` element inside one of the `<td>` cells of an outer table.

Table 68. Nested table inside table cell

HTML Input	DITA Output
<table> <tr> <td>Label</td> <td> <table> <tr>	<table frame="all" rowsep="1" colsep="1" id="table_outer"> <tgroup cols="2"> <colspec colnum="1" colname="col1"/> <colspec colnum="2" colname="col2"/> <tbody> <row>

Table 68. Nested table inside table cell (continued)

HTML Input	DITA Output
<td>Sub-item A</td> <td>Value 1</td> </tr> <tr> <td>Sub-item B</td> <td>Value 2</td> </tr> </table> </td> </tr> </table>	<entry>Label</entry> <entry> <table frame="all" rowsep="1" colsep="1" id="table_inner"> <tgroup cols="2"> <colspec colnum="1" colname="col1"/> <colspec colnum="2" colname="col2"/> <tbody> <row> <entry>Sub-item A</entry> <entry>Value 1</entry> </row> <row> <entry>Sub-item B</entry> <entry>Value 2</entry> </row> </tbody> </tgroup> </table> </entry> </row> </tbody> </tgroup> </table>

Result: The converter identifies the nested `<table>` element and preserves it as a CALS `<table>` inside the parent table cell. The inner table is wrapped in a `<p>` element inside the outer `<entry>` to separate the nested structure from the parent cell content.

Warning:
Nested tables increase complexity and may affect rendering, accessibility, and publishing output. Flatten the nested structure into a single table where possible.

Scenario 6 — List inside a table cell

Consider a table where the author has placed a `<ul>` or `<ol>` list inside one of the `<td>` cells.

Table 69. List inside a table cell

HTML Input	DITA Output

`<table>`
`  <tr>`
`  <th>Step</th>`
`  <th>Actions</th>`
`  </tr>`
`  <tr>`
`  <td>1</td>`
`  <td>`
`  <ul>`
`  <li>Download the package.</li>`
`  <li>Verify the checksum.</li>`
`  <li>Extract the archive.</li>`
`  </ul>`
`  </td>`
`  </tr>`
`</table>`
	`<simpletable>`
	`  <sthead>`
	`  <stentry>Step</stentry>`
	`  <stentry>Actions</stentry>`
	`  </sthead>`
	`  <strow>`
	`  <stentry>1</stentry>`
	`  <stentry>`
	`  <ul>`
	`  <li>Download the package.</li>`
	`  <li>Verify the checksum.</li>`
	`  <li>Extract the archive.</li>`
	`  </ul>`
	`  </stentry>`
	`  </strow>`
	`</simpletable>`

Result: The converter preserves the `<ul>` or `<ol>` list inside the table cell and maps it to the corresponding DITA `<ul>` or `<ol>` element inside `<stentry>`.

> **Tip:**
> Use lists inside table cells only when the list represents multiple related values for the same row. Avoid using lists inside tables for procedural steps. If the content is procedural, consider converting it to a DITA task with `<steps>`.

Scenario 7 — Image inside a table cell

Consider a table where the author has inserted an `<img>` element directly inside a `<td>` cell — for example, a status icon or screenshot thumbnail.

Table 70. Image inside a table cell

HTML Input	DITA Output

`<table>`
`  <tr>`
`  <th>Status</th>`
`  <th>Icon</th>`
`  <th>Description</th>`
`  </tr>`
`  <tr>`
`  <td>OK</td>`
`  <td>`
`  <img src="media/ok_icon.png" alt="OK icon"/>`
`  </td>`
`  <td>System running.</td>`
`  </tr>`
`  <tr>`
`  <td>Error</td>`
`  <td>`
`  <img src="media/err_icon.png" alt="Error icon"/>`
`  </td>`
`  <td>Error detected.</td>`
`  </tr>`
`</table>`
	`<simpletable>`
	`  <sthead>`
	`  <stentry>Status</stentry>`
	`  <stentry>Icon</stentry>`
	`  <stentry>Description</stentry>`
	`  </sthead>`
	`  <strow>`
	`  <stentry>OK</stentry>`
	`  <stentry>`
	`  <image href="media/ok_icon.png" placement="inline"/>`
	`  </stentry>`
	`  <stentry>System running.</stentry>`
	`  </strow>`
	`  <strow>`
	`  <stentry>Error</stentry>`
	`  <stentry>`
	`  <image href="media/err_icon.png" placement="inline"/>`
	`  </stentry>`
	`  <stentry>Error detected.</stentry>`
	`  </strow>`
	`</simpletable>`

Result: Inline `<img>` elements inside table cells are extracted and mapped to `<image placement="inline"/>` inside the corresponding `<stentry>`. The `src` attribute value is mapped to the `href` attribute on the DITA `<image>` element. For more information on image handling, see the Images and Graphics *(on page 67)* section.

Scenario 8 — Table spanning multiple pages

Consider a long reference table whose rendered output spans two or more printed pages.

Table 71. Image inside a table cell

HTML Input	DITA Output
<h2 id="status-indicator-lights"> Status Indicator Lights </h2> <table> <tr> <th>Light Pattern</th> <th>Color</th> <th>Status</th> </tr> <tr> <td>Solid</td> <td>Green</td> <td>Printer is ready.</td> </tr> <tr> <td>Flashing</td> <td>Orange</td> <td>Paper jam detected.</td> </tr> <tr> <td>Solid</td> <td>Red</td> <td>Ink level is critically low.</td> </tr> <tr> <td>Flashing</td> <td>Green</td> <td>Firmware update in progress.</td> </tr> <tr> <td>Solid</td> <td>Orange</td> <td>Ink level is low.</td>	<reference id="status_indicator_lights"> <title>Status Indicator Lights</title> <refbody> <simpletable> <sthead> <stentry>Light Pattern</stentry> <stentry>Color</stentry> <stentry>Status</stentry> </sthead> <strow> <stentry>Solid</stentry> <stentry>Green</stentry> <stentry>Printer is ready.</stentry> </strow> <strow> <stentry>Flashing</stentry> <stentry>Orange</stentry> <stentry>Paper jam detected.</stentry> </strow> <strow> <stentry>Solid</stentry> <stentry>Red</stentry> <stentry>Ink level is critically low.</stentry> </strow> <strow> <stentry>Flashing</stentry> <stentry>Green</stentry> <stentry>Firmware update in progress.</stentry> </strow> <strow> <stentry>Solid</stentry> <stentry>Orange</stentry> <stentry>Ink level is low.</stentry>

Table 71. Image inside a table cell (continued)

HTML Input	DITA Output
</tr> <tr> <td>Flashing</td> <td>Red</td> <td>Hardware error detected.</td> </tr> </table>	</strow> <strow> <stentry>Flashing</stentry> <stentry>Red</stentry> <stentry>Hardware error detected.</stentry> </strow> </simpletable> </refbody> </reference>

Result: The converter produces one `<simpletable>` or `<table>` that contains all rows from the original HTML table. No splitting or truncation occurs because of page breaks. Page break handling is managed by the DITA publishing pipeline, not by the converter.

Scenario 9 — Multiple consecutive tables with no paragraph between them

Consider a section where two or more `<table>` elements appear one after another in the HTML source with no `<p>` element between them.

Table 72. Consecutive tables, no paragraph in between

HTML Input	DITA Output
<h2 id="server-configuration"> Server Configuration </h2> <table> <tr> <th>Parameter</th> <th>Value</th> </tr> <tr> <td>Port</td> <td>8080</td>	<reference id="server_configuration"> <title>Server Configuration</title> <refbody> <simpletable> <sthead> <stentry>Parameter</stentry> <stentry>Value</stentry> </sthead> <strow> <stentry>Port</stentry> <stentry>8080</stentry>

Table 72. Consecutive tables, no paragraph in between (continued)

HTML Input	DITA Output
</tr> <tr> <td>Timeout</td> <td>60</td> </tr> </table> <table> <tr> <th>Parameter</th> <th>Value</th> </tr> <tr> <td>Max connections</td> <td>100</td> </tr> <tr> <td>Retry limit</td> <td>3</td> </tr> </table>	</strow> <strow> <stentry>Timeout</stentry> <stentry>60</stentry> </strow> </simpletable> <simpletable> <sthead> <stentry>Parameter</stentry> <stentry>Value</stentry> </sthead> <strow> <stentry>Max connections</stentry> <stentry>100</stentry> </strow> <strow> <stentry>Retry limit</stentry> <stentry>3</stentry> </strow> </simpletable> </refbody> </reference>

Result: Each HTML `<table>` element is converted independently into a separate DITA `<simpletable>` or `<table>` element. The converter does not automatically merge consecutive tables.

Tip:
If the tables are logically related, add a short bridging `<p>` element between them in the HTML source before conversion.

Scenario 10 — 2-column table in an API reference section

Consider a 2-column table inside a section classified as an API reference topic — for example, a parameter list or API attribute table where the heading contains a keyword such as "API", "parameters", or "reference".

Table 73. Two-column table, API reference

HTML Input

```
<h2 id="api-parameters">API Parameters</h2>

<table>
  <tr>
    <th>Parameter</th>
    <th>Description</th>
  </tr>
  <tr>
    <td>timeout</td>
    <td>Request timeout in seconds.</td>
  </tr>
  <tr>
    <td>retry_count</td>
    <td>Number of retry attempts.</td>
  </tr>
  <tr>
    <td>log_level</td>
    <td>Logging verbosity.</td>
  </tr>
</table>
```

DITA Output

```
<reference id="api_parameters">
  <title>API Parameters</title>
  <refbody>
    <properties>
      <prophead>
        <proptypehd>Parameter</proptypehd>
        <propvaluehd>Description</propvaluehd>
      </prophead>
      <property>
        <proptype>timeout</proptype>
        <propvalue>Request timeout in seconds.</propvalue>
      </property>
      <property>
        <proptype>retry_count</proptype>
        <propvalue>Number of retry attempts.</propvalue>
      </property>
      <property>
        <proptype>log_level</proptype>
        <propvalue>Logging verbosity.</propvalue>
      </property>
    </properties>
  </refbody>
</reference>
```

Result: The converter recognises the API reference context from the heading keyword and maps the 2-column `<table>` to `<properties>`. For general comparison tables, status tables, or multi-column reference data, the converter maps to `<simpletable>` or CALS `<table>` instead.

Chapter 12. Admonitions

This section covers Admonitions — paragraph-level notices that give safety-related, advisory, or supplementary information structural separation from the surrounding body content — and five scenarios: a standard named admonition, consecutive admonitions, an image placed inside an admonition, an admonition inside a table cell, and a bold inline label pattern. The converter maps each recognized admonition style directly to `<note type="[type]">`; for bold inline labels, content pattern recognition reads the opening text, removes the label, and maps the remaining content to the matching type.

Admonitions are paragraph-level notices that separate advisory, safety-related, or supplementary information from the surrounding body content. Admonitions identify information that requires immediate reader attention — safety warnings, usage restrictions, helpful tips, important prerequisites, and cautionary notices — and give it structural separation from the prose that surrounds it.

For readers and publishing systems, admonition apply stronger visual treatment or weight from tip through note through caution through warning through danger. Each one of them are distinct in admonition icons, background color, or text color.

In well-structured HTML, admonitions are commonly authored using `<div>`, `<aside>`, or `<p>` elements with recognizable class names or labels that indicate the type of advisory information.

The converter maps a paragraph with a recognized admonition style directly to `<note type="[type]">`. For a bold inline label, it applies content pattern recognition: it reads the opening text, removes the label, and maps the remaining text to the matching note type.

Important:
Use native HTML admonition structures, such as recognized class attributes or supported bold-label patterns. Do not simulate admonitions using bold text, colored paragraphs, or other visual formatting alone, as these patterns cannot be converted reliably into DITA `<note>` elements.

Table 74. Admonition patterns mapped to DITA

HTML Admonition Pattern	Purpose	DITA Note Type
`<div class="note"> or <p><strong>Note:</strong>`	General information	`<note>`
	Warn about damage, data loss,	`<note type="warning">`

Table 74. Admonition patterns mapped to DITA (continued)

HTML Admonition Pattern	Purpose	DITA Note Type
`<div class="warning"> or <p><strong>Warning:</strong>`	safety, or severe consequences	
`<div class="caution"> or <p><strong>Caution:</strong>`	Warn about possible problems or unexpected behavior	`<note type="caution">`
`<div class="danger"> or <p><strong>Danger:</strong>`	Warn about hazardous situations or serious injury	`<note type="danger">`
`<div class="important"> or <p><strong>Important:</strong>`	Highlight critical information	`<note type="important">`
`<div class="remember"> or <p><strong>Remember:</strong>`	Remind users about important recurring information	`<note type="remember">`
`<div class="attention"> or <p><strong>Attention:</strong>`	Draw attention to a specific item	`<note type="attention">`
`<div class="tip"> or <p><strong>Tip:</strong>`	Provide helpful advice or shortcuts	`<note type="tip">`
`<div class="restriction"> or <p><strong>Restriction:</strong>`	Describe limitations or unsupported behavior	`<note type="restriction">`
`<div class="Fast-Path"> or <p><strong>Fast-Path:</strong>`	Provide an alternate quicker approach	`<note type="fastpath">`

Table 74. Admonition patterns mapped to DITA (continued)

HTML Admonition Pattern	Purpose	DITA Note Type
`<div class="troubleshooting"> or <p><strong>Troubleshooting:</strong>`	Provide troubleshooting guidance	`<note type="trouble">`
`<div class="custom-type"> or unrecognised label`	Custom note type	`<note type="other" othertype="...">`

In Oxygen XML Text view

1. Edit the type attribute directly in the XML source. Locate the `<note>` tag.
2. Add or update the type attribute.

Scenario 1 — Image placed inside an admonition

Consider a element where the author has inserted an <img> element — for

Consider a `<div class="warning">` element where the author has inserted an `<img>` element — for example, a photograph of a hazardous component — directly inside the warning content.

Table 75. Image placed inside admonition

HTML Input	DITA Output
`<div class="warning"> <p> Do not touch the marked components. <img ... /> </p> </div>`	`<note type="warning"> Do not touch the marked components. <fig id="..."> <image placement="break" href="media/hazard_component.png" id="image_s1v_ymr_pjc"> <alt>an image</alt> </image> </fig> </note>`

Result: The converter extracts the `<img>` element and maps it to a DITA `<fig>` element containing `<image placement="break"/>`. The `<note>` element wraps both the warning text and the `<fig>` as a single block-level admonition.

Scenario 2 — Admonition inside a table cell

Consider a `<table>` where one of the `<td>` cells contains a `<div class="warning">` element rather than a plain `<p>` element.

Table 76. Admonition inside a table cell

HTML Input	DITA Output

`<table>`
`  <tr>`
`  <th>Setting</th>`
`  <th>Notes</th>`
`  </tr>`
`  <tr>`
`  <td>Production Mode</td>`
`  <td>`
`  <div class="warning">`
`  <p>`
`  Do not modify this value in production.`
`  </p>`
`  </div>`
`  </td>`
`  </tr>`
`</table>`
	`<simpletable>`
	`  <sthead>`
	`  <stentry>Setting</stentry>`
	`  <stentry>Notes</stentry>`
	`  </sthead>`
	`  <strow>`
	`  <stentry>Production Mode</stentry>`
	`  <stentry>`
	`  <note type="warning">`
	`  Do not modify this value in production.`
	`  </note>`
	`  </stentry>`
	`  </strow>`
	`</simpletable>`

Result: When an admonition element appears inside a `<td>` cell, the converter wraps the content in a `<note>` element and places it inside the corresponding DITA `<stentry>` or `<entry>`. The admonition does not break out of the table cell boundary — it is contained within the cell and rendered as inline note content within that cell.

Scenario 3 — Bold-label admonition pattern

Consider a <p> element where the author has formatted an admonition using a label

Consider a `<p>` element where the author has formatted an admonition using a `<strong>` label followed by the admonition text, rather than using a `<div>` with a class attribute.

Table 77. Bold-label admonition pattern

HTML Input	DITA Output

`<strong>Warning:</strong> Always back up your data before performing a firmware update.`	`<note type="warning">`
	`  Always back up your data before performing a firmware update.`
	`</note>`

Result: The converter detects the `<strong>` label pattern at the start of a `<p>` element and maps it to the appropriate `<note type="...">` element. The `<strong>` label text and separator (:) are stripped from the DITA output — only the admonition body text is preserved inside the `<note>` element.



Note:
The bold-label pattern is a fallback. Use class-based admonition elements such as `<div class=\"warning\">` for reliable conversion.

Chapter 13. Cross-References and Hyperlinks

This section covers Cross-References, Bookmarks and Hyperlinks — three distinct navigational mechanisms that connect related content within a document, across a documentation set, or to an external resource — and six scenarios: an internal cross-reference or bookmark, an external URL, a hyperlink with non-descriptive display text, a link list at the end of a section, and an internal link list used as a mini-TOC. The converter maps each structured link to a DITA `<xref>` element using @href, @scope, and @format to resolve the target correctly in the generated output

A **cross-reference,** a **bookmark**, and a **hyperlink**are three distinct mechanisms - each with a specific structural role:

- A **cross-reference** is an active, field-based link to a heading, figure, table, footnote, or bookmark within the same document or internal identifier (such as bookmark).
- A **bookmark**assigns a named identifier to a specific location in the document — a word, a phrase, a paragraph, or a position between characters. Bookmarks serve as anchor points that cross-references and hyperlinks can target.

> **Note:**
> Both cross-reference and/or bookmarks target updates automatically when the document is restructured or the sections are moved within the same document.

- A **hyperlink**stores an explicit address — a URL, an email address, a file path, or a sometimes same-document bookmark anchor. Unlike bookmark and cross reference, the hyperlink address is fixed at the time of creation.

> **Note:**
> Using the document's native link tools is the condition that produces a valid, resolvable <xref> element in the DITA output. A link typed as plain text produces no link structure and maps to a plain <p> element.

In HTML, links are created using the `<a>` element with an `href` attribute that identifies the target location. The converter evaluates the target of each HTML `<a>` element when generating DITA output and resolves each link type using three `<xref>` attributes:

Table 78. Xref attributes, purpose, and values

At-tribute	Purpose	Common Values
@href	The link target	Topic file path, URL, mailto: address
@scope	Relationship between source and target	local — same deliverable peer — same documentation set, different deliverable external — outside the documentation set
@for-mat	Format of the target resource	Cross-reference - dita — default for DITA topic references Hyperlink - html — external web pages; PDF — linked PDF files

Important:
Ensure all hyperlinks reference valid targets and use descriptive link text. The converter flags unresolvable links in the conversion report.

Scenario 1 — Internal link

Consider an HTML document where the author has used an anchor link to

Consider an HTML document where the author has used an `<a href="#prerequisites">` anchor link to reference a heading elsewhere in the same document.

Table 79. Internal link

HTML Input	DITA Output

`<p>`
`  For prerequisites, see`
`  <a href="#prerequisites">`
`  Prerequisites section`
`  </a>.`
`</p>`
`<p>`
`  Refer to`
`  <a href="configuration_guide.html">`
`  The Configuration Guide`
	`<p>`
	`  For prerequisites, see the`
	`  <xref href="t001_prerequisites.dita">`
	`  Prerequisites section`
	`  </xref>.`
	`</p>`
	`<p>`
	`  Refer to`
	`  <xref href="t002_configuration_guide.dita" format="dita">`
	`  The Configuration Guide`

Table 79. Internal link (continued)

HTML Input	DITA Output
. 	</xref>.

Result: If valid internal anchor links are present and resolve to a heading ID in the HTML package, the converter maps the `href` to the corresponding generated .dita topic file. If the anchor target cannot be resolved, the converter retains only the visible link text as plain text and flags the broken reference in the conversion report.

Scenario 2 — External URLs

Consider a `<p>` element that contains an `<a href="...">` element pointing to an external URL.

Table 80. Internal link

HTML Input	DITA Output
 See the release notes for details. 	 See <xref href="https://example.com/release-notes" format="html" scope="external"> the release notes </xref> for details.

Result: External hyperlinks are preserved. The `href` attribute receives the URL, scope="external" is added, and the format attribute is set based on the URL file extension or target type. The element content receives the visible link text.

Scenario 3 — Hyperlink with non-descriptive display text

Consider an <a> element where the author has used non-descriptive display text such as "click

Consider an `<a>` element where the author has used non-descriptive display text such as "click here", "here", or a bare URL as the link's visible label, rather than a phrase that describes the destination.

Table 81. Hyperlink with non-descriptive text

HTML Input	DITA Output
`Click <a href="https://example.com/docs">here</a> for the full API reference.`	`<p> See <xref href="https://example.com/docs" format="html" scope="external"> click here </xref> for the full API reference. </p> <!-- Display text "click here" preserved verbatim -->`

Result: The hyperlink is converted correctly into a DITA `<xref>` element with display text preserved.

Note:
Use descriptive hyperlink text such as "API reference" or "release notes" instead of "click here" or raw URLs. Non-descriptive link text reduces accessibility and makes DITA output harder to navigate.

Chapter 14. Math and Equations

This section covers Mathematical Equations — quantitative expressions that cannot be represented accurately through prose alone — and five scenarios: a block equation, an inline equation, an equation in a table cell, an equation in a list item, and an equation in a procedure step. The converter maps native `<math>` (MathML) markup to `<foreign>` containing embedded MathML; image-based equations map to `<image>` with a required `<alt>` element carrying the equation's accessible text description.

Mathematical equations communicate quantitative relationships that cannot be expressed accurately through prose alone. They are commonly used in engineering, scientific, financial, and technical documentation to represent formulas, calculations, algorithms, and symbolic expressions.

The mathematical equation can be inserted as:

- **MathType objects** - MathType is a third-party equation editor that embeds equations as OLE (Object Linking and Embedding) binary objects.
- **Image-based equations** - An equation represented as a PNG, EMF, or other image file — whether inserted directly or produced by a screenshot — carries no mathematical structure.
- **Reference-based equations** – An equation is stored separately as an external MathML file, which can be reused or maintained outside the topic.

In HTML, mathematical content may be authored as native MathML, rendered as images, or represented using LaTeX syntax through client-side rendering libraries.

The converter evaluates how each equation is represented in the HTML source before determining the generated DITA output. Native MathML is preserved and converted to the corresponding DITA MathML structure. Image-based equations are treated as standard images, while LaTeX expressions are converted only when the mathematical markup is available in the HTML source before conversion. The quality of the generated DITA output depends directly on the format in which mathematical content is authored.

The following table lists the types of equations and their corresponding DITA output:

Table 82. Equation types mapped to DITA

Mathematical Equation	DITA structure
Equation inside a sentence	`<equation-inline>`
Equation within a paragraph, list paragraph, and tables	`<equation-block>`
Equation with title/caption/reference need	`<equation-figure>`

Table 82. Equation types mapped to DITA (continued)

Mathematical Equation	DITA structure
Numbered equation	`<equation-number>` inside the equation structure
Actual mathematical notation	`<mathml>` with MathML markup
Reference/external equation link	`<mathmlref>`

Important:
Author mathematical expressions using native MathML. Image-based equations and client-side rendering libraries do not produce structured content during conversion.

Scenario 1 — Block equation

Consider a <math> element placed between two <p> elements as a standalone block equation.

Consider a `<math>` element placed between two `<p>` elements as a standalone block equation.

Table 83. Standalone block equation

HTML Input

```
<p>
  The transfer rate is:
</p>
<math
  xmlns="http://www.w3.org/1998/Math/MathML"
  display="block">
  <mrow>
    <mi>R</mi>
    <mo>=</mo>
    <mfrac>
      <mi>B</mi>
      <mi>T</mi>
    </mfrac>
  </mrow>
```

DITA Output

```
<p>The transfer rate is:</p>
<equation-block>
  <math xmlns="http://www.w3.org/1998/Math/MathML">
    <mrow>
      <mi>R</mi>
      <mo>=</mo>
      <mfrac>
        <mi>B</mi>
        <mi>T</mi>
      </mfrac>
    </mrow>
  </math>
</equation-block>
<p>
```

Table 83. Standalone block equation (continued)

HTML Input	DITA Output

`</math>`
`<p>`
`  where B is total bytes, and T is the elapsed time.`
`</p>`
	`  where B is total bytes, and T is the elapsed time.`
	`</p>`

Result: The converter detects the `<math display="block">` element and wraps it in a DITA `<equation-block>` element. The MathML content is preserved as-is inside the `<equation-block>`.

Scenario 2 — Inline equation

Consider a <math> element placed inline within a <p> element alongside surrounding text.

Consider a `<math>` element placed inline within a `<p>` element alongside surrounding text.

Table 84. Inline equation within text

HTML Input	DITA Output

`<p>`
`  The formula`
`  <math xmlns="http://www.w3.org/1998/Math/MathML">`
`    <mrow>`
`      <mi>E</mi>`
`      <mo>=</mo>`
`      <mi>m</mi>`
`      <msup>`
`        <mi>c</mi>`
`        <mn>2</mn>`
`      </msup>`
`    </mrow>`
`  </math>`
`  defines energy equivalence.`
`</p>`
	`<p>`
	`  The formula`
	`  <equation-inline>`
	`    <mathml>`
	`      <math xmlns="http://www.w3.org/1998/Math/MathML">`
	`        <mrow>`
	`          <mi>E</mi>`
	`          <mo>=</mo>`
	`          <mi>m</mi>`
	`          <msup>`
	`            <mi>c</mi>`
	`            <mn>2</mn>`
	`          </msup>`
	`        </mrow>`
	`      </math>`
	`    </mathml>`
	`  </equation-inline>`

Table 84. Inline equation within text (continued)

HTML Input	DITA Output
	defines energy equivalence. </p>

Result: The converter detects the `<math>` element positioned inline within a `<p>` element and preserves the MathML content inline within the surrounding `<p>` element. No `<equation-block>` wrapper is produced for inline equations.

Note:
The generated MathML structure is preserved from the HTML source. Validate mathematical output after conversion to ensure rendering accuracy in HTML, PDF, and downstream publishing systems.

Scenario 3— Equation in a table cell

Consider a `<table>` where one of the `<td>` cells contains a `<math>` element — for example, a reference table listing formulas alongside their descriptions.

Table 85. Equation in a table cell

HTML Input	DITA Output
<table> <tr> <th>Formula</th> <th>Description</th> </tr> <tr> <td> <math xmlns="http://www.w3.org/1998/Math/MathML"> <mrow> <mi>R</mi> <mo>=</mo> <mfrac> <mi>B</mi>	<simpletable> <sthead> <stentry>Formula</stentry> <stentry>Description</stentry> </sthead> <strow> <stentry> <equation-inline> <mathml> <math xmlns="http://www.w3.org/1998/Math/MathML"> <mrow> <mi>R</mi> <mo>=</mo>

Table 85. Equation in a table cell (continued)

HTML Input	DITA Output
<mi>T</mi> </mfrac> </mrow> </math> </td> <td>Transfer rate</td> </tr> </table>	<mfrac> <mi>B</mi> <mi>T</mi> </mfrac> </mrow> </math> </mathml> </equation-inline> </stentry> <stentry>Transfer rate</stentry> </strow> </simpletable>

Result: The converter processes the `<math>` element within the `<td>` cell and converts it to an `<equation-inline>` element containing the MathML content, placed inside the corresponding DITA `<stentry>`. The equation is treated as inline content within the cell and does not break out of the table cell boundary.

Scenario 4 — Equation in a list item

Consider a <ul> list where one of the <li> items contains a <math> element inline

Consider a `<ul>` list where one of the `<li>` items contains a `<math>` element inline within the item text.

Table 86. Equation in a list item

HTML Input	DITA Output
    Use the formula <math xmlns="http://www.w3.org/1998/Math/MathML"> <mrow> <mi>A</mi> <mo>=</mo> <mi>π</mi> <msup>	 Use the formula <equation-inline> <mathml> <math xmlns="http://www.w3.org/1998/Math/MathML"> <mrow> <mi>A</mi> <mo>=</mo>

Table 86. Equation in a list item (continued)

HTML Input	DITA Output
<mi>r</mi> <mn>2</mn> </msup> </mrow> </math> to calculate area. Round results to two decimal places. 	<mi>π</mi> <msup> <mi>r</mi> <mn>2</mn> </msup> </mrow> </math> </mathml> </equation-inline> to calculate area. Round results to two decimal places.

Result: When a `<math>` element is embedded inline within a `<li>` item, the converter preserves it as an `<equation-inline>` element inside the DITA `<li>`. The equation is treated as part of the list item's inline content and remains within the list structure.

Scenario 5 — Equation in a procedure step

Consider a task topic where a numbered step contains a <math> element placed as

Consider a task topic where a numbered `<ol><li>` step contains a `<math>` element placed as a block equation after the step command text.

Table 87. Equation in a procedure step

HTML Input	DITA Output
    Calculate the load using: <math xmlns="http://www.w3.org/1998/Math/MathML"	<task id="calculate_load"> <taskbody> <steps> <step> <cmd>Calculate the load using:</cmd>

Table 87. Equation in a procedure step (continued)

HTML Input	DITA Output
display="block"> <mrow> <mi>P</mi> <mo>=</mo> <mi>I</mi> <mi>V</mi> </mrow> </math> Record the result. 	<info> <equation-block> <mathml> <math xmlns="http://www.w3.org/1998/Math/MathML"> <mrow> <mi>P</mi> <mo>=</mo> <mi>I</mi> <mi>V</mi> </mrow> </math> </mathml> </equation-block> </info> </step> <step> <cmd>Record the result.</cmd> </step> </steps> </taskbody> </task>

Result: When a `<math display="block">` element is placed inside a `<li>`item after the step command text, the converter places it inside `<step><info>` as an `<equation-block>`, binding it to the parent step. The equation adopts the procedural step's context and is rendered as part of that step's supplementary content.

Chapter 15. Additional Reference/Appendix

Topic Type Classification

is the process of assigning a heading-delimited section of HTML content to a specific DITA

Topic type classification is the process of assigning a heading-delimited section of HTML content to a specific DITA topic type. Unlike DITA, HTML does not explicitly identify a section as conceptual, procedural, reference, or troubleshooting content. Instead, the structure and purpose of a section are inferred from the heading and the HTML elements that follow it until the next heading of the same or higher level.

Accurate classification matters because every DITA topic type generates a distinct body element — `<conbody>`, `<taskbody>`, `<refbody>`, `<troublebody>`, or `<body>` — and each body element supports a specific content structure required for that topic type. A help system that filters search results by topic type, a content management system that applies different review workflows to tasks versus references, or a PDF generator that styles procedures with numbered callouts, all depend on the topic type being assigned correctly at conversion time.



Important:
The HTML-to-DITA converter evaluates two complementary signals when determining the topic type. The converter first evaluates the wording of the heading. It then analyzes the dominant HTML structure beneath that heading. This structure could include paragraphs, ordered lists, tables, or other recurring content patterns. When both signals indicate the same type of content, the converter generates the corresponding DITA topic and body element. It then restructures the HTML content into the child elements required by that topic type.

The following table summarizes the primary content patterns that the converter uses when determining the generated DITA topic type:

Table 88. Content patterns mapped to topic type

Content Pattern After Heading	DITA Topic Type	DITA Body Element
Descriptive paragraphs, definitions, and overviews	Concept	`<conbody>`
Numbered lists `<ol>`, procedural instructions	Task	`<taskbody>`

Table 88. Content patterns mapped to topic type (continued)

Content Pattern After Heading	DITA Topic Type	DITA Body Element
Tables, parameter lists, lookup data	Reference	`<refbody>`
Problem / Cause / Solution structure	Troubleshooting	`<troublebody>`
No dominant classification signal	Generic	`<body>`

Concept Topic

A concept section is recognized by a descriptive or noun-phrase heading such as "Print Quality,"

A concept section is recognized by a descriptive or noun-phrase heading such as "Print Quality," "Connection Modes," or "How the Print Spooler Works," paired with a body of explanatory paragraphs, definition tables, or comparison tables. No ordered list as the dominant body structure.

When these signals align, the converter generates a:

- `<concept>` element with `<conbody>`.
- The opening paragraph maps to `<shortdesc>` and subsequent paragraphs map to `<p>` inside `<conbody>`.
- Tables in a concept topic always map to `<simpletable>` unless a `<caption>` element or merged cells are present, in which case the table routes to a CALS `<table>`.
- A 2-column table in a concept topic never maps to `<properties>` — that element is only generated in a Reference topic with an API keyword context.

The following scenarios demonstrate how different HTML heading and content patterns influence Concept topic classification:

Scenario 1 — Prose as a Concept

A section titled "How the Print Spooler Works" opens with descriptive paragraphs explaining background information.

A section titled "How the Print Spooler Works" opens with descriptive paragraphs explaining background information. The heading contains no action verb, and the body contains no numbered list or table.

Table 89. Prose classifies heading as concept

HTML Input	DITA Output

`<h2 id="how-print-spooler-works">How the Print Spooler Works</h2>`
`<p>The print spooler manages print jobs sent to the printer queue.</p>`
`<p>Jobs are processed in the order they are received unless priority settings are applied.</p>`
	`<concept id="how_print_spooler_works">`
	`<title>How the Print Spooler Works</title>`
	`<shortdesc>The print spooler manages print jobs sent to the printer queue.</shortdesc>`
	`<conbody>`
	`<p>The print spooler manages print jobs sent to the printer queue.</p>`
	`<p>Jobs are processed in the order they are received unless priority settings are applied.</p>`
	`</conbody>`
	`</concept>`

Result: Descriptive prose with no numbered list and no table triggers concept classification. All `<p>` elements map to `<p>` inside `<conbody>` in document order. The first `<p>` element is also extracted as `<shortdesc>`.

Scenario 2 — Graphical Concept

Consider a section titled "Ink Tank Assembly" with an <h2> , a <p> describing the

Consider a section titled "Ink Tank Assembly" with an `<h2>`, a `<p>` describing the assembly, and an image of the ink tank area wrapped in a `<figure>` element with a `<figcaption>` reading "Figure 1: Ink tank and cap locations."

Table 90. Image classifies heading as concept

HTML Input	DITA Output

`<h2 id="ink-tank-assembly">Ink Tank Assembly</h2>`
`<p>`
`The ink tank assembly holds the ink supply for each color channel.`
`</p>`
	`<concept id="ink_tank_assembly">`
	`<title>Ink Tank Assembly</title>`
	`<conbody>`
	`<p>The ink tank assembly holds the ink supply for each color channel.</p>`

Table 90. Image classifies heading as concept (continued)

HTML Input	DITA Output
<figure> <figcaption> Figure 1: Ink tank and cap locations </figcaption> </figure>	<fig> <title>Ink tank and cap locations</title> <image href="media/ink_tank_area.png"> <alt>Ink tank and cap locations</alt> </image> </fig> </conbody> </concept>

Result: An image accompanied by descriptive prose does not change the topic type. Because the heading and surrounding text remain descriptive — explaining what something is, with no imperative verb or sequence of actions — the section classifies as concept. The `<figure>`/`<figcaption>` pair maps to `<fig>` with the caption text as its `<title>`, nested inside `<conbody>`, and the `alt` attribute maps to the `<alt>` element inside `<image>`.

Task Topic

A task section is recognized by an imperative heading such as "Install the Driver" or

A task section is recognized by an imperative heading such as "Install the Driver" or "Replace the Toner Cartridge," paired with an ordered list (`<ol>`) as the primary body structure. Both signals must be present. An imperative heading without an `<ol>`, or an `<ol>` without an imperative heading, does not reliably trigger task classification.

When these signals align, the converter generates a:

- `<task>` element with `<taskbody>`.
- The opening paragraph maps to both `<shortdesc>` and `<context>`.
- Each `<li>` in the primary `<ol>` maps to `<step><cmd>`.
- A `<ul>` list or introductory paragraph appearing before the first `<ol>` maps to `<prereq>`. A paragraph or second `<ol>` appearing after the final step maps to `<postreq>`.
- A `<figure>` or `<img>` nested inside a `<li>` maps to `<step><info>` as supporting content for that step.
- A nested `<ol>` inside a `<li>` maps to `<substeps>`, with each nested `<li>` mapping to `<substep><cmd>`.
- A nested `<ul>` inside a `<li>` maps to `<info><ul>` as supporting information unless the items are short and mutually exclusive, in which case it maps to `<choices>`.

The following scenarios demonstrate how different HTML task patterns are interpreted during HTML-to-DITA conversion:

Scenario 1 — Simple Task

A section titled "Install the Printer Driver" opens with a context sentence followed by a

A section titled "Install the Printer Driver" opens with a context sentence followed by a numbered list of steps. The heading contains an action verb, and the primary body content is a numbered list.

Table 91. Simple Task

HTML Input	DITA Output

`<h2 id="install-printer-driver">Install the printer driver</h2>`
`<p>Follow these steps to install the printer driver on your system.</p>`
`<ol>`
`  <li>Download the driver package from the support portal.</li>`
`  <li>Run the installer and accept the license agreement.</li>`
`  <li>Restart your system when prompted.</li>`
`</ol>`
	`<task id="install_printer_driver">`
	`  <title>Install the Printer Driver</title>`
	`  <shortdesc>Follow these steps to install the printer driver on your system.</shortdesc>`
	`  <taskbody>`
	`  <context>`
	`  <p>Follow these steps to install the printer driver on your system.</p>`
	`  </context>`
	`  <steps>`
	`  <step>`
	`  <cmd>Download the driver package from the support portal.</cmd>`
	`  </step>`
	`  <step>`
	`  <cmd>Run the installer and accept the license agreement.</cmd>`
	`  </step>`
	`  <step>`
	`  <cmd>Restart your system when prompted.</cmd>`
	`  </step>`
	`  </steps>`
	`  </taskbody>`
	`</task>`

Result: The heading action verb ("Install") combined with an `<ol>` as the primary body content triggers task classification. The introductory `<p>` maps to `<context>` and each `<li>` maps to `<step><cmd>`.

Scenario 2 — Graphical Task

Consider a numbered procedure where the author has placed an <img> element with a <figcaption>

Consider a numbered procedure where the author has placed an `<img>` element with a `<figcaption>` inside a `<figure>` inside an `<li>` element — for example, to show a button as part of a step.

Table 92. Graphical Task

HTML Input	DITA Output

`<h2 id="locate-the-power-button">Locate the Power Button</h2>`
`<ol>`
`  <li>`
`  Locate the power button.`
`  <figure>`
`  <img src="media/power_button.png" alt="Power button"/>`
`  <figcaption>`
`  Power button location`
`  </figcaption>`
`  </figure>`
`  </li>`
`</ol>`
	`<task id="locate_the_power_button">`
	`  <title>Locate the Power Button</title>`
	`  <taskbody>`
	`  <steps>`
	`  <step>`
	`  <cmd>Locate the power button.</cmd>`
	`  <info>`
	`  <fig>`
	`  <title>Power button location</title>`
	`  <image href="media/power_button.png">`
	`  <alt>Power button</alt>`
	`  </image>`
	`  </fig>`
	`  </info>`
	`  </step>`
	`  </steps>`
	`  </taskbody>`
	`</task>`

Result: The converter maps the `<figure>` and `<figcaption>` inside the `<li>` to a `<fig>` structure, placed inside `<step><info>`, not inside `<cmd>`. The task body remains a single contiguous `<steps>` block.

Reference Topic

A reference section is recognized by a noun-phrase heading containing a lookup keyword such as

A reference section is recognized by a noun-phrase heading containing a lookup keyword such as "Supported File Formats," "Error Codes," or "API Parameters," paired with a table or parameter list as the dominant body structure. The heading keyword is the primary classification signal. A section containing only a table but no lookup keyword in the heading may be classified as Concept or Generic depending on the surrounding context.

When these signals align, the converter generates a `<reference>` element with `<refbody>`. The opening paragraph maps to `<shortdesc>`. Any introductory prose appearing before the first table is wrapped in a `<section>` element inside `<refbody>` rather than placed directly in `<refbody>`. A 2-column table in a section whose heading contains an API keyword such as "API," "parameters," "fields," or "attributes" maps to `<properties>`. The same 2-column table in a reference section without an API keyword maps to `<simpletable>`.

The following scenarios demonstrate how different HTML reference patterns are interpreted during HTML-to-DITA conversion:

Scenario 1 — Standard Multi - Column Table Reference

A section titled "Printer Configuration Parameters" contains a brief introductory sentence and a multi-column table of configuration settings. The heading contains the keyword "Parameters", and a lookup table dominates the body.

Table 93. Standard multi-column table reference

HTML Input	DITA Output

`<h2 id="printer-config-parameters">Printer configuration parameters</h2>`
`<p>The following parameters control printer behaviour.</p>`
`<table>`
`  <thead>`
`  <tr>`
`  <th scope="col">Parameter</th>`
`  <th scope="col">Value</th>`
`  <th scope="col">Description</th>`
`  </tr>`
`  </thead>`
`  <tbody>`
	`<reference id="printer_config_parameters">`
	`  <title>Printer Configuration Parameters</title>`
	`  <shortdesc>The following parameters control printer behaviour.</shortdesc>`
	`  <refbody>`
	`  <section>`
	`  <p>The following parameters control printer behaviour.</p>`
	`  </section>`
	`  <simpletable frame="all" relcolwidth="1* 1* 1*" id="simpletable_zhd_2wg_pjc">`
	`  <sthead>`
	`  <stentry>Parameter</stentry>`

Table 93. Standard multi-column table reference (continued)

HTML Input	DITA Output
<tr> <td>copies</td> <td>1–99</td> <td>Number of copies to print.</td> </tr> </tbody> </table>	<stentry>Value</stentry> <stentry>Description</stentry> </sthead> <strow> <stentry>copies</stentry> <stentry>1–99</stentry> <stentry>Number of copies to print.</stentry> </strow> </simpletable> </refbody> </reference>

Result: The heading keyword "Parameters" combined with a multi-column lookup table triggers reference classification. Non-table prose before the table is wrapped in a `<section>` inside `<refbody>`. The multi-column HTML `<table>` maps to `<simpletable>`. Column header text maps to plain `<stentry>` elements.

Scenario 2 — 2 — Column Table in API Reference Section

A section titled "POST /api/print-job" documents a REST API endpoint. The topic is classified as reference because the heading contains an API keyword and the body contains a 2-column parameter table.

Table 94. Two-column table in API reference section

HTML Input	DITA Output
<h2 id="post-api-print-job">POST /api/print-job</h2> Submits a new print job to the queue. <table> <tr> <th>Field</th> <th>Description</th> </tr> <tr>	<reference id="post_api_print_job"> <title>POST /api/print-job</title> <shortdesc>Submits a new print job to the queue.</shortdesc> <refbody> <properties frame="all" id="properties_z4h_mxg_pjc"> <prophead> <proptypehd>Field</proptypehd> <propvaluehd>Description</propvaluehd> </prophead>

Table 94. Two-column table in API reference section (continued)

HTML Input	DITA Output
<td>document_id</td> <td>ID of document to print.</td> </tr> <tr> <td>copies</td> <td>Number of copies (default: 1).</td> </tr> </table>	<property> <proptype>document_id</proptype> <propvalue>ID of document to print</propvalue> </property> <property> <proptype>copies</proptype> <propvalue>Number of copies (default: 1)</propvalue> </property> </properties> </refbody> </reference>

Result: The 2-column HTML `<table>` maps to `<properties>` because the topic is classified as API reference. The same table in a concept topic would produce `<simpletable>`.

Note:
A 2-column table maps to `<properties>`only when the topic is classified as reference with an API keyword context. Outside of that context, a 2-column HTML `<table>` always maps to `<simpletable>`.

For a complete summary of how HTML table structures map to DITA output elements, see Tables — Overview *(on page 78)*.

Troubleshooting Topic

A troubleshooting section is recognized by a problem-oriented heading such as "The Printer Does Not

A troubleshooting section is recognized by a problem-oriented heading such as "The Printer Does Not Power On" or "Print Jobs Fail to Complete," paired with a body that presents a condition, a cause, and a corrective action. The converter looks for bold labels in the form `Cause:` and `Remedy:` in the body. Both the heading and the body labels must be present. A problem-oriented heading without cause and remedy labels causes the converter to classify the section as Task or Generic depending on the body structure.

When these signals align, the converter generates a `<troubleshooting>` element with `<troublebody>`. The problem heading maps to `<title>` and the condition text maps to `<condition>`. Each cause label and its associated text maps to `<cause>` inside a `<troubleSolution>` block. Each remedy label and its associated

steps maps to `<remedy>` inside the same `<troubleSolution>` block, with each `<li>` mapping to `<step><cmd>`. When a section contains multiple cause-and-remedy pairs, the converter generates one `<condition>` at the top of `<troublebody>` and a separate `<troubleSolution>` for each pair. The condition is not repeated for each solution.

The following scenarios demonstrate how different HTML troubleshooting patterns are interpreted during HTML-to-DITA conversion:

Scenario 1 — Clearly defined problem, cause and remedy

Consider an HTML document containing a troubleshooting section with a clearly defined problem, its cause,

Consider an HTML document containing a troubleshooting section with a clearly defined problem, its cause, and the corresponding remedy. The following example shows how the converter transforms this structure into a DITA `<troubleshooting>` topic.

Table 95. Troubleshooting: problem, cause, remedy

HTML Input	DITA output

`<h2>The Printer Does Not Power On</h2>`
`<p><b>Cause:</b> The power cable is disconnected.</p>`
`<p><b>Remedy:</b></p>`
`<ol>`
`  <li>Connect the power cable securely.</li>`
`  <li>Press the power button.</li>`
`</ol>`
	`<troubleshooting id="trouble_rzn_xrj_rjc">`
	`  <title>The Printer Does Not Power On</title>`
	`  <troublebody>`
	`  <condition>`
	`  <title>Condition</title>`
	`  <p>The printer does not power on.</p>`
	`  </condition>`
	`  <troubleSolution>`
	`  <cause>`
	`  <title>Cause</title>`
	`  <p>The power cable is disconnected.</p>`
	`  </cause>`
	`  <remedy>`
	`  <title>Remedy</title>`
	`  <steps>`
	`  <step>`
	`  <cmd>Connect the power cable securely.</cmd>`
	`  </step>`

Table 95. Troubleshooting: problem, cause, remedy (continued)

HTML Input	DITA output
	<step> <cmd>Press the power button.</cmd> </step> </steps> </remedy> </troubleSolution> </troublebody> </troubleshooting>

Result: The heading "The Printer Does Not Power On" states a problem, and the body uses `<b>Cause:</b>` and `<b>Remedy:</b>` labels triggering a Troubleshooting topic. The heading text becomes the `<condition>`, since it states the observable problem. The cause text maps to `<cause>`, and the numbered remedy steps map to `<remedy><steps>`. The cause and remedy are wrapped together in a single `<troubleSolution>`.

Scenario 2 — Multiple causes for one problem

Consider an HTML document containing one condition followed by multiple causes and corresponding remedies.

Consider an HTML document containing one condition followed by multiple causes and corresponding remedies.

Table 96. Troubleshooting: multiple causes, one problem

HTML Input	DITA output
The Printer Does Not Print Cause 1: Paper tray is empty. Remedy: Load paper into the paper tray. Cause 2: Printer is offline. Remedy: 	<troubleshooting id="trouble_rzn_xrj_rjc"> <title>The printer does not print</title> <troublebody> <condition> <title>Condition</title> The printer does not print </condition> <troubleSolution> <cause id="cause_jb3_nwj_rjc">

Table 96. Troubleshooting: multiple causes, one problem (continued)

HTML Input	DITA output
Reconnect the printer to the network. 	<title>Cause 1</title> Paper tray is empty. </cause> <remedy id="remedy_kb3_nwj_rjc"> <title>Remedy</title> <steps id="steps_lb3_nwj_rjc"> <step> <cmd>Load paper.</cmd> </step> </steps> </remedy> </troubleSolution> <troubleSolution> <cause id="cause_qb3_nwj_rjc"> <title>Cause 2</title> Printer is offline. </cause> <remedy id="remedy_vhq_dwj_rjc"> <title>Remedy</title> <steps id="steps_usn_2wj_rjc"> <step> <cmd>Reconnect printer.</cmd> </step> </steps> </remedy> </troubleSolution> </troublebody> </troubleshooting>

Result: Each cause-and-remedy pair produces its own `<troubleSolution>`, while the shared problem heading maps to a single `<condition>` at the top of `<troublebody>`.

Generic Topic

A Generic topic is the converter's fallback classification when the heading and body content together

A Generic topic is the converter's fallback classification when the heading and body content together do not provide sufficient signals to identify a Concept, Task, Reference, or Troubleshooting pattern. The most common triggers are a descriptive heading followed only by a `<ul>` list, mixed body content where no single element type is dominant, a heading verb and body structure that conflict, or a section with no introductory paragraph and no table or ordered list. A `<ul>` list on its own never triggers task classification — task classification requires both an imperative heading and an `<ol>` as the primary body content.

When Generic classification is assigned, the converter generates a standard `<topic>` element with `<body>`. All supported HTML elements are mapped in document order without applying the specialized structures associated with other topic types. When no introductory paragraph is present, the converter writes an empty `<shortdesc>` and flags it at WARNING severity in the conversion report. Generic topics are also flagged as candidates for manual reclassification if the content appears to match a more specific topic type.

The following scenarios demonstrate how different HTML patterns result in Generic classification during HTML-to-DITA conversion:

Scenario 1 — Generic Topic - No Dominant Classification Signal

A section titled "What's New in Version 4.2" has an introductory sentence and a bullet

A section titled "What's New in Version 4.2" has an introductory sentence and a bullet list of features. The heading contains no classification keyword, and the body does not contain any numbered list or table.

Table 97. Generic topic, no clear signal

HTML Input	DITA Output

`<h2 id="whats-new-v4-2">What's new in version 4.2</h2>`
`<p>Version 4.2 introduces the following improvements.</p>`
`<ul>`
`  <li>Faster print spooling.</li>`
`  <li>Improved Wi-Fi reconnect logic.</li>`
`  <li>Reduced energy use in standby.</li>`
`</ul>`
	`<topic id="whats_new_in_version_4_2">`
	`  <title>What's New in Version 4.2</title>`
	`  <shortdesc>Version 4.2 introduces the following improvements.</shortdesc>`
	`  <body>`
	`  <p>Version 4.2 introduces the following improvements.</p>`
	`  <ul>`
	`  <li>Faster print spooling.</li>`
	`  <li>Improved Wi-Fi reconnect logic.</li>`
	`  <li>Reduced energy use in standby.</li>`
	`  </ul>`

Table 97. Generic topic, no clear signal (continued)

HTML Input	DITA Output
	</body> </topic>

Result: No classification signal is strong enough, so the topic becomes generic `<topic>` with plain `<body>`.

Scenario 2 — Generic Topic - Empty <shortdesc> (No Introductory Paragraph)

Scenario 2 — Generic Topic - Empty <shortdesc> (No Introductory Paragraph) A section titled "Known

A section titled "Known Limitations in Version 4.2" opens directly with a bulleted list — no introductory paragraph precedes it.

Table 98. Generic topic, empty shortdesc

HTML Input	DITA Output
<h2 id="known-limitations-v4-2">Known limitations in version 4.2</h2> - Auto-duplex not supported above 200 gsm. - Wi-Fi Direct limited to four simultaneous connections. - Scheduled jobs not preserved across firmware updates. 	<topic id="known_limitations_v4_2"> <title>Known Limitations in Version 4.2</title> <shortdesc/> <!-- empty — no intro para found --> <!-- validator: WARNING empty_shortdesc --> <body> - Auto-duplex not supported above 200 gsm. - Wi-Fi Direct limited to four simultaneous connections. - Scheduled jobs not preserved across firmware updates. </body> </topic>

Result: Empty `<shortdesc>` and a validator WARNING are produced. To fix: add an introductory `<p>` element directly after the `<h2>` in the HTML source and rerun conversion or add the `<shortdesc>` text manually in the DITA output.

Index

A

absolute
76

admonition
92, 95

Admonition
42, 61

Admonitions
10

application
45

Application
45

architecture
70

assembly
110

Audience
7

Authentication
40

author
18

available
9

B

backup
43

Backup
43

before
55

between
50, 61

block
102, 106

Bold
31

bullet
54

bullets
51

button
73, 113

C

caption
71

cause
116, 117, 118

causes
118

class
92

classification
119

click
99

Code
30, 31

column
78

concept
26, 109

Concept
109

condition
116

configuration
59

Configure

36
connection
64
content
12, 13, 18, 20, 23, 25, 108
Conversion
6
converter
7, 12

D

definition
62
Definition
62
deployment
35, 46, 58, 61
Deployment
58
descriptive
99
device
64, 65
dialog
69
display
99
DITA
6, 7, 8, 28, 67, 108
ditamap
23
Ditamap
11, 15
document
17, 75, 97
draft
44
driver
112

E

element
26, 29, 30, 31, 59, 60, 64, 70, 71, 94, 102, 103, 105
elements
8
emphasis
29
Empty
121
Environment
35, 46
equation
101, 102, 103
Equation
105, 106
equations
101
Error
86

F

firmware
42
folder
15

G

general
29
Generic
119, 120, 121
Getting
7
green
74
guide
7

H

heading
20, 21, 111, 116

Heading
22

headings
23

height
69

hierarchy
14

HTML
6, 7, 9, 11, 11, 12, 13, 15, 15, 17, 23, 25, 26, 75, 78, 98

I

image
58, 67, 75, 75, 76, 110

images
67

imperative
111

index
14

information
10, 92

inline
26, 28, 30, 103, 105

Inline
28

inside
29, 60, 73, 74, 75, 86, 86, 94

install
112

Install
34

Installation
21, 22, 51

installer
50

Intended
7

Internal
98

items
33, 54, 55

K

keyword
113

L

label
95

later
55, 56

level
22, 33, 39, 40, 51, 54, 56, 57

lifecycle
35

Linux
34

List
50, 86

lists
33, 33, 62

Lists
33

locations
71, 110

M

Maps
8

MathML
101

metadata
17

Migration
9

N

navigation
14, 31
nested
38, 40, 56, 57, 111
Next
69
Numbered
34

O

Open
45
ordered
33, 38

P

package
75
Package
11
paragraph
25
Paragraph
59
parts
26
pattern
65, 95
position
76
power
73, 113, 117
Power
113
prerequisites
98
present
23
print
109
printer
26, 112
Printer
15, 21
procedure
106
prolog
18
Prose
109
Purpose
7

R

reader
10
reference
97
release
42
remedy
117, 118
result
9, 12
review
37

S

Save
29
section
113
separator
65
sequence

31
server
36, 39
Server
29
settings
69
shortdesc
121
spooler
109
Started
7
status
74
steps
36, 39, 43
system
31

T

table
60, 78, 86, 86, 113
target
97
Three
57
title
15
topic
11, 13, 20, 108, 109, 119, 120
topichead
23
Typical
11

U

update
42

V

Version
120

W

warning
94
width
69, 69, 70
workflow
37, 42, 44, 46
Workflow
37, 38, 42, 44